

더조은 컴퓨터 아카데미 (종로)

통합 채용 공고 검색과 지원 관리 플랫폼

팀원 | 이예영 권현경 안보현 이가은

멘토 | 장기태

TABLE OF CONTENTS

목차

1	프로젝트 개요	3p
2	프로젝트 팀 구성 및 역할	8p
3	프로젝트 수행 절차 및 방법	9p
4	프로젝트 수행 경과	10p
5	자체 평가 의견	93p

1. 프로젝트 개요

✓ 프로젝트 주제

다양한 채용 사이트의 공고를 주기적으로 통합 수집하고, Elasticsearch 기반의 고속 복합 검색 및 Kibana 시각화 대시보드 기능을 제공하는 채용 정보 통합 웹 플랫폼

✓ 선정 배경

- 채용 플랫폼이 다양해지면서 구직자가 여러 사이트를 일일이 검색해야 하는 번거로움이 존재
- ‘백엔드 신입’, ‘Java’, ‘서울 강남’, ‘재택 가능’ 등 복합 조건 검색이 기존 플랫폼에서 어려움
- 수집된 채용 정보를 한눈에 파악할 수 있는 시각화 도구나 통계 기능도 부재

✓ 기획 의도

- 여러 채용 사이트의 공고를 한 곳에서 통합적으로 확인
- 직무, 지역, 재택 가능 여부, 기술스택 등 다양한 조건 조합 검색
- 직무별/지역별 채용 비율, 마감 추이 등 대시보드 시각화
- OpenAI 기반 지능형 챗봇을 통해 사용자 친화적인 정보 탐색 지원

1. 프로젝트 개요

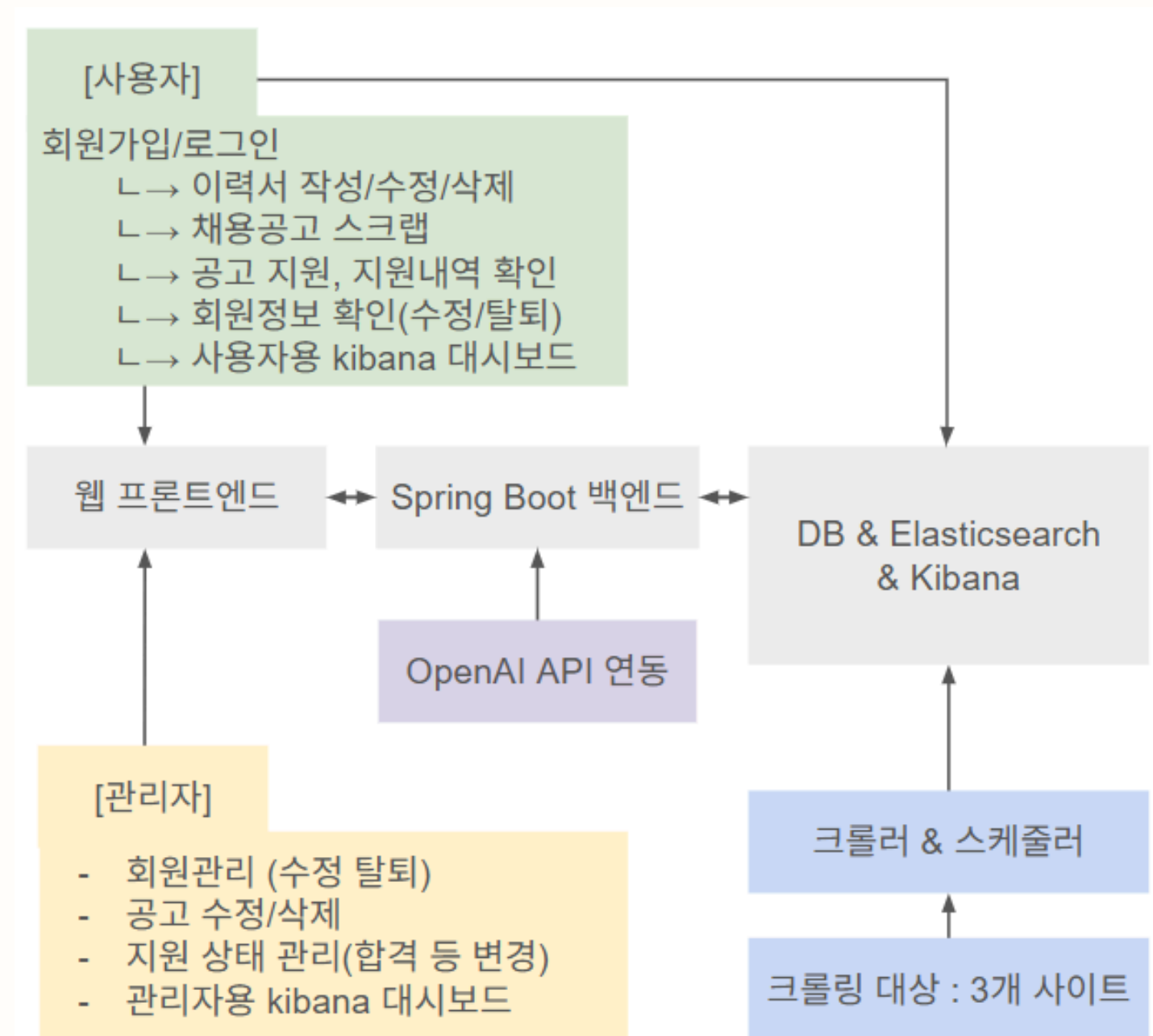
✓ 프로젝트 컨셉

채용공고 수집 + 복합 검색 + 이력서 작성/지원 + 시각화 + 챗봇 Q&A를 통합한 올인원 구직지원 플랫폼

✓ 훈련 과정과의 연관성

웹 백엔드 개발, REST API, DB 설계, 크롤링, 보안 (Spring Security), 검색엔진 등
훈련과정에서 배운 핵심 기술을 실습 기반으로 종합 구현

✓ 구현 내용



✓ 개발환경

구분	핵심 기술
Backend	Java 17, Spring Boot 3.3.12, Spring Data JPA
Frontend(View)	Thymeleaf, Thymeleaf Layout Dialect, JS
Database	MariaDB (JPA 연동)
검색엔진	Elasticsearch 8.12.2 (Spring Data Elasticsearch)
크롤링	JSoup, Selenium
보안	Spring Security
시각화	Kibana
챗봇(AI 연동)	OpenAI API
개발 도구	IntelliJ, Gradle

1. 프로젝트 개요

✓ 프로젝트 목적

- 잡O리아, 잡플O닛, 원O드 등에서 공고 자동 수집
- Elasticsearch로 고속 검색 및 복합 조건 필터링 구현
- Kibana로 데이터 시각화 제공
- OpenAI 챗봇 연동으로 대화형 기능 지원

✓ 프로젝트 진행 일정 2025년 6월 23일 ~ 2025년 7월 16일



1. 프로젝트 개요

✓ 활용 방안

- 서비스 대상 : 구직자, 인사담당자, 채용 플랫폼 운영자
- 사용 시나리오 : 채용 공고 탐색, 스크랩, 이력서 제출 등
- 기술적 활용 : Elasticsearch 기반 복합 검색, Kibana 대시보드로 공고 통계 등

✓ 기대효과

- 사용자 측면 : 구직 효율 향상, 정보 접근성 증대
- 관리자/기업 측면 : 채용 트렌드 분석, 타겟 마케팅 가능
- 비즈니스/실무 활용성 : 플랫폼 확장성, 데이터 기반 기능 개선 가능

2. 프로젝트 팀 구성 및 역할

이름	역할	담당 업무
이예영	팀장	데이터 수집, DB제작, 데이터 전처리, 지원 현황 관리, 스크랩, 관리자 채용공고, 관리자 지원 관리, 검색, 정렬, 페이지네이션
권현경	팀원	데이터 수집, 프론트엔드, 이력서 관리, 이미지 업로드, AI챗봇
안보현	팀원	데이터 수집, 프론트엔드, 채용공고, 사용자 지원관리, 정렬, 페이지네이션, 대시보드
이가은	팀원	데이터 수집, 데이터 전처리, 로그인/회원가입, 회원정보수정 및 탈퇴(사용자/관리자), 정렬, 페이지네이션

3. 프로젝트 수행 절차 및 방법

구분	기간	활동	비고
사전 기획	6/23	프로젝트 주제 선정, 요구 기능 도출, 기존 채용 사이트 분석	아이디어 구체화, UI/UX 설계 포함
데이터 수집	6/23~6/26	Jokorea, Wanted, 잡플레닛 공고 크롤링(Selenium & Jsoup 활용)	스케줄러로 자동 수집 설정
데이터 전처리	6/24~6/27	불필요한 공고 필터링, 마감일 변환, 중복 제거, 로고 경로 저장, mariaDB에 저장	엔티티 설계 및 Spring JPA 활용
웹 서비스 구축	6/30~7/8	Spring Boot 기반 서비스 구현, 회원가입/로그인, 이력서 작성, 공고 지원 등	Thymeleaf, Spring Security 등 사용
검색/시각화 구현	7/8~7/?	Elasticsearch에 인덱싱, 복합 조건 검색 API 구현, Kibana 시각화 대시보드 구현	필터, 정렬 등 고급 검색 기능 포함
챗봇 구현	7/8~7/11	사용자 입력 처리, OpenAI API 연동, 실시간 응답 출력 UI 구현	OpenAI API 사용, 응답 처리, UI 구현
기능 테스트 및 개선	7/14~7/16	기능별 디버깅	마무리 테스트 및 UI 정리 포함
총 개발 기간	24일	-	-

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 크롤링 사이트 3개 선정

잡코O아, 잡플O닛, 원O드 등 사이트별 구조 차이로 인해 크롤링 방식 차별화

JOBKOREA

채용정보

(주)일화 관심기업

(주)일화 신입 / 경력 사원 모집

지원자격	근무조건	기업정보
경력: 신입-경력 직무별 상이	고용형태: 정규직	산업(업종): 기타 비알코올 음료 제조업
학력: 고졸이상 직무별 상이	급여: 회사내규에 따름	사원수: 360명
스킬: Microsoft Office, 운전면허	지역: 서울시 강동구, 중북 청주시 청원구, 대전전지역, 대구전지역, 부산전지역, 울산전지역, 광주전지역, 전북전지역 지도	설립년도: 1973년 (53년차)
우대: 기본우대	시간: 주5일 (월~금) 08:30~17:30	기업형태: 중견기업 (비상장)
장애편 +		홈페이지: ilhwa.co.kr

[기업정보 >](#) [진행중인 채용보기 >](#)

[홈페이지 지원](#) [☆ 스크랩](#)

상세요강 | 접수기간/방법 | 자소서항목 | 기업정보 | 추천공고 **NEW** | 대전·세종·충청 지역채용관 >

4대보험 가입정보 - 국민연금, 건강보험, 고용보험, 산재보험

Jobplanet

총 999+건 장학도순

Android SDK 개발자 (Kotlin/Next.js)
(주)버즈빌 | 서울
사내문화: 4.2

프론트엔드 엔지니어
(주)로민
복지 및 급여: 3.9

ASP.NET 중급개발자
오엠티아이코리아(주)
승진 기회 및 가능성: 3.5

웹 그래픽(F/E) 개발자 (5년 ~ 10년)
(주)데이터메이커
업무와 삶의 균형: 3.5

웹 그래픽(F/E) 개발자 (11년 ~ 15년)
(주)데이터메이커
업무와 삶의 균형: 3.5

Android SDK 개발자(Kotlin/Next.js)
(주)버즈빌 | 서울

상세 | 리뷰 | 면접

요약

마감일: 2025.07.31 **D-21**

직무: 안드로이드 개발, 프론트엔드 개발

경력: 3 ~ 15년

고용형태: 정규직

근무지역: 서울

스킬: java, kotlin, react, rxjava, Gradle, Next.js, coroutine

기업 이미지

wanted

Alwayz

레브잇 · 서울 관악구 · 경력 2년 이상

Software Engineer(Backend)

[응답률](#) [높음](#)

포지션 상세

레브잇은 가격을 비교할 필요도 없이 100% 고객신뢰로 경제적인 소비의 가치를 전달하는 모두가 매일 사용하는 초저가 커머스 플랫폼, '올웨이즈'를 운영하고 있습니다.

[합류하게 될 팀]

- Software Engineer (백엔드)는 목적조직으로 Group에 소속되어 다양한 Maker와 함께 그로스/커머스 제품을 개발하고, 기능조직으로 Backend Chapter에 소속되어 높은 개발 생산성과 개발문화를 만들어가고 있습니다.
- 레브잇의 모든 구성원은 누군가에게 업무를 지시하는 것이 아니라, 본인 영역 직무전문성을 발휘하고 동료들과 협업하여 비즈니스 목표달성에 기여하고 있습니다.

합격보상 지원자, 추천인 각 현금 50만원

* 위 합격보상은 정규직 기준이며, 계약직은 지원자와 추천인 각각 현금 10만원으로 달라질 수 있어요.

[지원하기](#)

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 잡O리아 : 정적 페이지 기반 Jsoup 크롤링

- Jsoup: HTML 파싱 및 DOM 요소 추출을 위한 라이브러리
- HTTP POST 요청 : AJAX 기반으로 채용공고 리스트를 요청(비동기 로딩 구조 대응)

```
String ajaxUrl = "https://www.jobkorea.co.kr/Recruit/Home/_GI_List/";
Connection.Response response = Jsoup.connect(ajaxUrl)
    .userAgent("Mozilla/5.0")
    .header(name: "Referer", value: "https://www.jobkorea.co.kr/recruit/joblist?menucode=local&localorder=1")
    .header(name: "X-Requested-With", value: "XMLHttpRequest")
    .data("tabType", "local")
    .data("localorder", "1")
    .data("page", String.valueOf(page))
    .method(Connection.Method.POST)
    .timeout(millis: 10000)
    .execute();

Document doc = Jsoup.parse(response.body());
Elements rows = doc.select(cssQuery: "tr.devloopArea");

for (Element row : rows) {
    Element a = row.selectFirst(cssQuery: "td.tplTit strong a");
    if (a == null) continue;

    String title = a.text();
    String link = "https://www.jobkorea.co.kr" + a.attr(attributeKey: "href");

    try {
        Document detail = Jsoup.connect(link)
            .userAgent("Mozilla/5.0")
            .timeout(millis: 20000).get();

        Recruitment job = new Recruitment();
        job.setTitle(title);
        job.setCompany(getText(detail, selector: "div.header > span.coName"));
```


4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 잡플O닛 : 동적 렌더링, Selenium 기반 크롤링

- Selenium WebDriver(ChromeDriver) : JavaScript 기반 동적 콘텐츠 로딩 처리
--> JobPlanet은 채용공고가 스크롤 할 때마다 동적으로 추가됨(AJAX+JS 기반)
- JavaScriptExecutor : 무한 스크롤 방식 대응
- WebDriver +DOM 탐색 : 상세 페이지 내 텍스트 요소 수집

```
driver.get("https://www.jobplanet.co.kr/job");
Thread.sleep( millis: 3000);

for (int i = 0; i < 30; i++) {
    ((JavascriptExecutor) driver).executeScript( script: "window.scrollTo(0, document.body.scrollHeight);");
    Thread.sleep( millis: 1000);
}

List<WebElement> cards = driver.findElements(By.cssSelector("a.group.block.medium"));
List<String> links = new ArrayList<>();
for (WebElement card : cards) {
    String href = card.getAttribute( name: "href");
    if (href != null && !href.isEmpty()) links.add(href);
}

for (int idx = 0; idx < Math.min(links.size(), 200); idx++) {
    String link = links.get(idx);
    driver.get(link);
    Thread.sleep( millis: 2000);

    Map<String, String> summary = new HashMap<>();
    List<WebElement> items = driver.findElements(By.cssSelector("dl.recruitment-summary_dli > *"));
    for (int i = 0; i + 1 < items.size(); i += 2) {
        summary.put(items.get(i).getText().trim(), items.get(i + 1).getText().trim());
    }

    Recruitment job = new Recruitment();
    job.setTitle(getText(driver, By.className("ttl")));
    job.setCompany(getText(driver, By.cssSelector("span.company_name a")));
```


4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 원O드: 동적 렌더링, Selenium 기반 크롤링 +상세 버튼 조작 필요

- Explicit Wait(WebDriverWait) : 버튼 클릭 가능한 시점까지 대기(상세 보기 더 보기)
-->상세 페이지에 더 보기 버튼이 있어 클릭해야 크롤링을 할 수 있음

- 이외는 잡플O닛과 동일

• 버전 관리: Git
• 모니터링 및 로깅: Datadog, Sentry
• 협업 도구 : Slack, Jira
• AI 개발 툴 : Claude Code, Cursor, Copilot, ChatGPT 등

상세 정보 더 보기

```
driver.get("https://www.wanted.co.kr/wdlist?country=kr&job_sort=job.latest_order&years=-1&locations=all");
Thread.sleep( millis: 3000);
for (int i = 0; i < 20; i++) {
    ((JavascriptExecutor) driver).executeScript( script: "window.scrollTo(0, document.body.scrollHeight);");
    Thread.sleep( millis: 1000);
}

Set<String> urls = new LinkedHashSet<>();
for (WebElement el : driver.findElements(By.tagName("a"))) {
    String href = el.getAttribute( name: "href");
    if (href != null && href.contains("/wd/")) {
        urls.add(href);
        if (urls.size() >= 200) break;
    }
}
for (String link : urls) {
    driver.get(link);
    Thread.sleep( millis: 2000);

    try {
        WebElement more = new WebDriverWait(driver, Duration.ofSeconds(3))
            .until(ExpectedConditions.elementToBeClickable(
                By.xpath( xpathExpression: "//button[.//span[contains(text(), '상세 정보 더 보기')]]")));
        ((JavascriptExecutor) driver).executeScript( script: "arguments[0].click();", more);
        Thread.sleep( millis: 1000);
    } catch (Exception ignored) {}

    Recruitment job = new Recruitment();
    job.setTitle(getText(driver, By.cssSelector("h1.wds-58fmok")));
    job.setCompany(getText(driver, By.cssSelector("a.JobHeader_JobHeader__Tools__Company__Link_NoBQ
```

4. 프로젝트 수행 경과

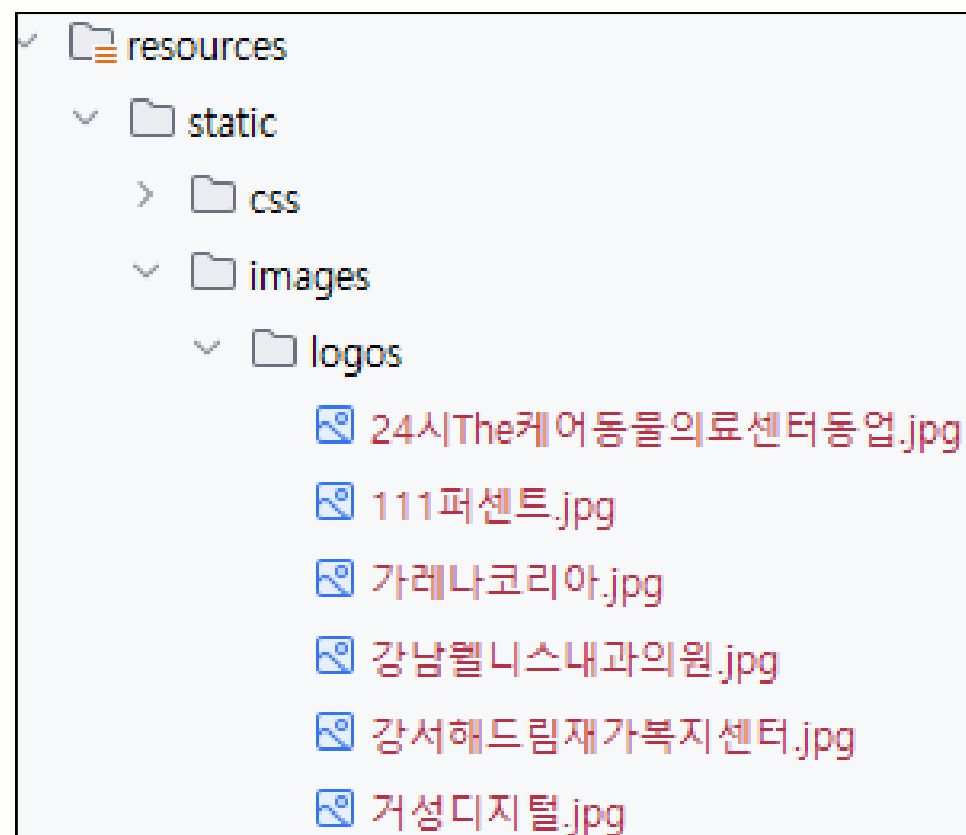
채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 로고 이미지 경로 지정

잡O리아의 회사 로고 저장 및 로컬 경로 설정 부분

```
Element logoEl = detail.selectFirst(cssQuery: "img#cologo");
String originLogoUrl = (logoEl != null) ? logoEl.absUrl(attributeKey: "src") : "";
String company = job.getCompany();
String localLogoPath = saveCompanyLogo(company, originLogoUrl);
job.setLogoUrl(localLogoPath);
```



✓ 로고 이미지 다운로드 및 저장

회사 로고 저장 및 로컬 경로 설정 부분

```
private String saveCompanyLogo(String company, String originUrl) { 3 usages  @tjoeunJava
    if (company == null || company.isEmpty() || originUrl == null || originUrl.isEmpty()) {
        return "";
    }
    try {
        File dir = new File(logoSaveDir);
        if (!dir.exists()) dir.mkdirs();
        String safeCompany = company.replaceAll(regex: "[^a-zA-Z0-9가-힣]", replacement: "");
        String fileName = safeCompany + ".jpg";
        File outFile = new File(dir, fileName);
        URL url = new URL(originUrl);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestProperty("User-Agent", "Mozilla/5.0");
        conn.setConnectTimeout(5000);
        conn.setReadTimeout(5000);
        conn.connect();
        try (InputStream in = conn.getInputStream()) {
            Files.copy(in, outFile.toPath(), StandardCopyOption.REPLACE_EXISTING);
        }
        return "/images/logos/" + fileName;
    } catch (Exception e) {
        System.out.println("[saveCompanyLogo 오류] " + e.getMessage());
        return "";
    }
}
```

4. 프로젝트 수행 경과

채용공고 수집과 Elasticserach 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 사이트별 방식 요약

사이트	상세 페이지 접근 방식	데이터 추출 방식	비고
JOBKOREA	정적 HTML => Jsoup	각 항목별 CSS 선택자	tr.devloopArea, div.header 등
JobPlanet	동적 렌더링 => Selenium	WebDriver + CSS 선택	스크롤 필요
Wanted	동적 렌더링 => Selenium	WebDriver + XPath 사용	스크롤, 상세 클릭 필요

✓ 전체 파싱

```
public void runCrawler() {  
    List<Recruitment> allJobs = new ArrayList<>();  
    allJobs.addAll(crawlJobKorea());  
    allJobs.addAll(crawlJobPlanet());  
    allJobs.addAll(crawlWanted());  
}
```

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → **전처리** → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 필수 항목 검사

공고명, 회사명, 주요업무 중 하나라도 없으면 제외

```
List<Recruitment> filteredJobs = allJobs.stream()
    .filter( Recruitment job -> isEmpty(job.getTitle()) &&
        isEmpty(job.getCompany()) &&
        isEmpty(job.getResponsibilities()))
    .collect(Collectors.toList());
```

✓ 메모리 기준 중복 제거 (title + company)

같은 공고명, 회사명 의 공고가 메모리 내 중복되는 경우 제거

```
Map<String, Recruitment> map = new LinkedHashMap<>();
for (Recruitment job : jobs) {
    String key = job.getTitle() + "|" + job.getCompany();
    map.putIfAbsent(key, job);
}
```

✓ DB 기준 중복 제거 (title+company+deadline)

DB 내 저장된 기존 공고와 완전히 일치하는 항목 제외

```
List<Recruitment> newJobs = uniqueJobs.stream()
    .filter( Recruitment job -> !repository.existsByTitleAndCompanyAndDeadline(
        job.getTitle(), job.getCompany(), job.getDeadline()))
    .collect(Collectors.toList());
```

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → **전처리** → DB 저장 → Elasticsearch 색인 → 검색 API 구현

✓ 전체 전처리 로직 적용

```
List<Recruitment> cleanedJobs = preprocessJobs(newJobs);
```

```
private List<Recruitment> preprocessJobs(List<Recruitment> jobs) { 1 usage  👤 tjoeunJava +1 *
    String defaultLogo = "/images/tjoeun.jpg";
    return jobs.stream()
        .peek( Recruitment job -> {
            if (job.getDeadline() == null) job.setDeadline(LocalDate.of( year: 9999, month: 12, dayOfMonth: 31));
            if (job.getQualifications() == null || job.getQualifications().isBlank())
                job.setQualifications("경력무관");
            if (job.getLogoUrl() == null || job.getLogoUrl().isBlank())
                job.setLogoUrl(defaultLogo);
            if (job.getPreferred() == null || job.getPreferred().isBlank())
                job.setPreferred("없음");
            if (job.getBenefits() == null || job.getBenefits().isBlank())
                job.setBenefits("회사내규에 따름");
            if (job.getLocation() == null || job.getLocation().isBlank())
                job.setLocation("홈페이지 참조");
            if (job.getSalary() == null || job.getSalary().isBlank())
                job.setSalary("회사내규에 따름");
            if (job.getEmploymentType() == null || job.getEmploymentType().isBlank())
                job.setEmploymentType("면접 후 결정");
        })
        .collect(Collectors.toList());
}
```

컬럼명	뜻	결측처리
id	idx	-
title	광고명	(행 삭제)
company	회사명	(행 삭제)
deadline	마감일	9999.12.31
qualifications	자격요건	경력무관
logoUrl	회사로고	/images/tjoeun.jpg
responsibilities	주요업무	(행 삭제)
preferred	우대사항	없음
benefits	복리후생	회사내규에 따름
location	근무지역	홈페이지 참조
salary	급여	회사내규에 따름
employment_type	고용형태	면접 후 결정

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → **DB 저장** → Elasticsearch 색인 → 검색 API 구현

✓ mariaDB DB 저장

UnifiedJobCrawlerService

- 크롤링 후 저장

```
int savedCount = 0;
for (Recruitment job : cleanedJobs) {
    try {
        Recruitment saved = repository.save(job);
        recruitmentSyncService.save(saved);
        savedCount++;
    } catch (Exception e) {
        System.out.println("[저장 실패] 중복 또는 오류: " + job.getTitle());
        e.printStackTrace();
    }
}

System.out.println("크롤링 및 저장 완료: 총 " + savedCount + "건 저장됨");
```



RecruitmentSyncService

- 앱이 시작될 때 (초기 한 번만) 또는 수동 호출 시, DB에 있는 모든 공고를 전체 동기화

```
public void syncAllToElasticsearch() { 2 usages  Ⓜ bhjava *
    List<Recruitment> recruitments = recruitmentRepository.findAllWithFavorites();
    System.out.println("불러온 DB 데이터 수: " + recruitments.size());

    int count = 0;
    for (Recruitment r : recruitments) {
        try {
            RecruitmentDocument doc = mapToDocument(r);
            System.out.println("색인할 문서: " + doc);
            recruitmentSearchRepository.save(doc);
            count++;
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    System.out.println("전체 ES 동기화 완료 - 총 색인 건수: " + count);
}
```

UserRecruitment와 유사하게 진행

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → **Elasticsearch 색인** → 검색 API 구현

✓ Elasticsearch 색인

RecruitmentSyncService

- 앱이 시작될 때 (초기 한 번만) 또는 수동 호출 시, DB에 있는 모든 공고를 전체 동기화

```
@Transactional 2 usages  ⚙ tjoeunJava
public Recruitment save(Recruitment recruitment) throws IOException {
    Recruitment saved = recruitmentRepository.save(recruitment);
    RecruitmentDocument doc = mapToDocument(saved);

    esClient.index( Builder<Object> i -> i
        .index(INDEX_NAME)
        .id(String.valueOf(doc.getRecruitmentIdx()))
        .document(doc)
    );

    return saved;
}
```

```
private RecruitmentDocument mapToDocument(Recruitment recruitment) { 2 usages  ⚙ tjoeunJava +3
    String combined = Optional.ofNullable(recruitment.getTitle()).orElse( other: "" ) + " " +
        Optional.ofNullable(recruitment.getResponsibilities()).orElse( other: "" );

    String jobKeywords = String.join( delimiter: " ", KoreanNounExtractor.extractNouns(combined));

    int scrapCount = favoriteRepository.countByRecruitment(recruitment);

    return RecruitmentDocument.builder()
        .recruitmentIdx(recruitment.getRecruitmentIdx())
        .title(recruitment.getTitle())
        .company(recruitment.getCompany())
        .deadline(safeFormatDeadline(recruitment.getDeadline()))
        .qualifications(recruitment.getQualifications())
        .logoUrl(recruitment.getLogoUrl())
        .responsibilities(recruitment.getResponsibilities())
        .preferred(recruitment.getPreferred())
        .benefits(recruitment.getBenefits())
        .location(recruitment.getLocation())
        .salary(recruitment.getSalary())
        .employmentType(recruitment.getEmploymentType())
        .createdAt(
            Optional.ofNullable(recruitment.getCreatedAt())
                .orElse(LocalDateTime.now())
        )
        .combinedContent(String.join( delimiter: " ",
            Optional.ofNullable(recruitment.getQualifications()).orElse( other: "" ),
            Optional.ofNullable(recruitment.getResponsibilities()).orElse( other: "" ),
            Optional.ofNullable(recruitment.getPreferred()).orElse( other: "" ),
            Optional.ofNullable(recruitment.getBenefits()).orElse( other: "" ),
            Optional.ofNullable(recruitment.getSalary()).orElse( other: "" ),
            Optional.ofNullable(recruitment.getEmploymentType()).orElse( other: "" )
        )
        .scrapCount(scrapCount)
        .jobKeywords(jobKeywords)
        .build();
}
```

RecruitmentSyncService

- Elasticsearch에 저장하기 위한 검색 최적화 형태로 변환하는 함수

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → **Elasticsearch 색인** → 검색 API 구현

✓ Elasticsearch 색인

RecruitmentDocument

- Elasticsearch 저장 대상

```
@JsonIgnoreProperties(ignoreUnknown = true) 29 usages  ⚙️ tjoenJava +2
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Document(indexName = "recruitments", createIndex = false)
@Data
public class RecruitmentDocument {

    @Id
    private Long recruitmentIdx;

    @Field(type = FieldType.Text, analyzer = "ngram_analyzer", searchAnalyzer = "ngram_analyzer")
    private String title;

    @Field(type = FieldType.Text, analyzer = "ngram_analyzer", searchAnalyzer = "ngram_analyzer")
    private String company;

    private String deadline;

    @Field(type = FieldType.Text)
    private String qualifications;

    @Field(type = FieldType.Keyword)
    private String logoUrl;
```

KIBANA 확인

GET /recruitments/_mapping

```
{
  "recruitments": {
    "mappings": {
      "properties": {
        "_class": {
          "type": "keyword",
          "index": false,
          "doc_values": false
        },
        "benefits": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "combinedContent": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            },
            "ngram": {
              "type": "text",
              "analyzer": "ngram_analyzer"
            }
          },
          "analyzer": "korean_custom"
        },
        "company": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            },
            "ngram": {
              "type": "text",
              "analyzer": "ngram_analyzer"
            }
          },
          "analyzer": "korean_custom"
        },
        "createdAt": {
          "type": "date",
          "format": "strict_date_optional_time"
        },
        "deadline": {
          "type": "date",
          "format": "strict_date_optional_time"
        },
        "employmentType": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "jobKeywords": {
          "type": "text",
          "fields": {
            "ngram": {
              "type": "text",
              "analyzer": "ngram_analyzer"
            }
          },
          "analyzer": "korean_custom",
          "field_data": true
        },
        "location": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            },
            "ngram": {
              "type": "text",
              "analyzer": "ngram_analyzer"
            }
          },
          "analyzer": "korean_custom"
        },
        "logoUrl": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "preferred": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "qualifications": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "recruitmentIdx": {
          "type": "long"
        },
        "responsibilities": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "salary": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "scrapCount": {
          "type": "long"
        },
        "title": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            },
            "ngram": {
              "type": "text",
              "analyzer": "ngram_analyzer"
            }
          },
          "analyzer": "korean_custom"
        }
      }
    }
  }
}
```


4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → **검색 API 구현**

✓ 검색 API 구현

RecruitmentSearchService.searchJobs – 검색 쿼리 로직

```
public Page<RecruitmentDocument> searchJobs( 2 usages 8 tjoeunJava +2
    String title,
    String content,
    String region,
    String company,
    String startDate,
    String endDate,
    String sortOrder,
    int page,
    int pageSize
) throws IOException {

    SearchRequest.Builder builder = new SearchRequest.Builder()
        .index(INDEX_NAME)
        .from(value: (page - 1) * pageSize)
        .size(pageSize);

    builder.query( Builder q -> q.bool( Builder b -> {

        if (title != null && !title.isBlank()) {
            String escapedTitle = escapeForWildcard(title);
            b.must( Builder m -> m.wildcard( Builder w -> w
                .field( value: "title")
                .value("*" + escapedTitle + "*")
            ));
        }

        if (content != null && !content.isBlank()) {
            String escapedContent = escapeForWildcard(content);
            b.must( Builder m -> m.wildcard( Builder w -> w
                .field( value: "combinedContent")
                .value("*" + escapedContent + "*")
            ));
        }

        if (region != null && !region.isBlank()) {
            String escapedRegion = escapeForWildcard(region);
            b.must( Builder m -> m.wildcard( Builder w -> w
                .field( value: "location")
                .value("*" + escapedRegion + "*")
            ));
        }
    })
}
```

```
if (company != null && !company.isBlank()) {
    String escapedCompany = escapeForWildcard(company);
    b.must( Builder m -> m.wildcard( Builder w -> w
        .field( value: "company")
        .value("*" + escapedCompany + "*")
    ));
}

// 날짜 범위 필터 (deadline)
// 날짜 필터 + 상시채용
if ((startDate != null && !startDate.isBlank()) || (endDate != null && !endDate.isBlank())) {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss");

    String formattedStart = null;
    String formattedEnd = null;

    if (startDate != null && !startDate.isBlank()) {
        formattedStart = LocalDate.parse(startDate).atStartOfDay().format(formatter);
    }
    if (endDate != null && !endDate.isBlank()) {
        formattedEnd = LocalDate.parse(endDate).atTime(hour: 23, minute: 59, second: 59).format(formatter);
    }

    String finalStart = formattedStart;
    String finalEnd = formattedEnd;

    b.filter( Builder f -> f.bool( Builder bool -> bool
        .should( Builder s -> s.range( Builder r -> {
            r.field( value: "deadline");
            if (finalStart != null) r.gte(JsonData.of(finalStart));
            if (finalEnd != null) r.lte(JsonData.of(finalEnd));
            return r;
        }))
        .should( Builder s -> s.term( Builder t -> t
            .field( value: "deadline")
            .value("9999-12-31T00:00:00")
        ))
    ));
}

return b;
});
```

```
// 정렬
switch (sortOrder) {
    case "all":
        builder.sort( Builder s -> s.field( Builder f -> f.field( value: "recruitmentIdx").order(SortOrder.Asc)));
        break;
    case "deadline_asc":
        builder.sort( Builder s -> s.field( Builder f -> f.field( value: "deadline").order(SortOrder.Asc)));
        break;
    case "deadline_desc":
        builder.sort( Builder s -> s.field( Builder f -> f.field( value: "deadline").order(SortOrder.Desc)));
        break;
    case "scrap_desc":
        builder.sort( Builder s -> s.field( Builder f -> f.field( value: "scrapCount").order(SortOrder.Desc)));
        break;
    case "scrap_asc":
        builder.sort( Builder s -> s.field( Builder f -> f.field( value: "scrapCount").order(SortOrder.Asc)));
        break;
    default:
        builder.sort( Builder s -> s.field( Builder f -> f.field( value: "createdAt").order(SortOrder.Desc)));
        break;
}

SearchRequest request = builder.build();

SearchResponse<RecruitmentDocument> response = esClient.search(request, RecruitmentDocument.class);

List<RecruitmentDocument> docs = response.hits().hits().stream().map(Hit::source).collect(Collectors.toList());

long total = response.hits().total().value();

return new PageImpl<>(docs, PageRequest.of(pageNumber: page - 1, pageSize), total);
}
```

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → **검색 API 구현**

✓ 검색 API 구현

```
// 검색 키워드 존재 시, 로그만 저장 (ES 검색 결과는 안 씀)
String combinedKeyword = Stream.of(title, content)
    .filter( String s -> s != null && !s.isBlank())
    .collect(Collectors.joining( delimiter: " "));

if (!combinedKeyword.isBlank()) {
    try {
        if (userId != null) {
            // 먼저 관리자 여부 판단
            boolean isAdmin = userRepository.findById(userId) Optional<Users>
                .map( Users user -> user.getUserRole() == UserRole.ADMIN) Optional<B
                .orElse( other: false);

            // 일반 사용자일 경우만 로그 저장 메서드 호출
            if (!isAdmin) {
                recruitmentSearchService.search(combinedKeyword, Long.valueOf(userId));
            }
        } else {
            // 비로그인 사용자도 로그 저장 (userId 없이)
            recruitmentSearchService.search(combinedKeyword, userId: null);
        }
    } catch (IOException e) {
        System.out.println("Elasticsearch 검색 로그 저장 실패: " + e.getMessage());
    }
}
```

EmpService

- 사용자의 검색 조건에 따라 Elasticsearch에서 채용공고를 검색하고, 결과를 DTO로 변환해 페이지징된 목록으로 반환

```
try {
    Page<RecruitmentDocument> esPage = recruitmentSearchService.searchJobs(
        title, content, region, company, startDate, endDate, sortOrder, page, pageSize
    );

    // 문서를 DTO로 변환
    List<RecruitmentDTO> dtoList = esPage.getContent().stream()
        .map( RecruitmentDocument doc -> RecruitmentDTO.builder()
            .recruitmentIdx(doc.getRecruitmentIdx())
            .title(doc.getTitle())
            .company(doc.getCompany())
            .deadline(parseDeadline(doc.getDeadline()))
            .scrapCount(doc.getScrapCount() != null ? doc.getScrapCount() : 0)
            .logoUrl(doc.getLogoUrl())
            .build()
        ).toList();

    return new PageImpl<>(dtoList, esPage.getPageable(), esPage.getTotalElements());
} catch (IOException e) {
    System.out.println("Elasticsearch 공고 검색 실패: " + e.getMessage());
    return Page.empty();
}
```

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → **검색 API 구현**

✓ 검색 API 구현

EmplController

- 채용공고 목록 조회

KIBANA 확인

```
GET /recruitments/_search
{
  "query": {
    "match": {
      "title": "간호사"
    }
  }
}
```

```
Page<RecruitmentDTO> jobPage = emplService.getJobPage(
    page, pageSize, sortOrder,
    title, content, region, company, startDate, endDate, userIdx);
```

```
{
  "_index": "recruitments",
  "_id": "9",
  "_score": 4.2422724,
  "_source": {
    "_class": "com.tjoeun.elasticsearch.document.RecruitmentDocument",
    "recruitmentIdx": 9,
    "title": "간호사, 간호조무사, 응급구조사 채용합니다/분당 서현역 도보2분 거리",
    "company": "서현365의원",
    "deadline": "2025-08-14T00:00:00",
    "qualifications": "경력 학력무관",
    "logoUrl": "/images/tjoeun.jpg",
    "responsibilities": "간호조무사",
    "preferred": "없음",
    "benefits": "회사내규에 따름",
    "location": "경기도 성남시 분당구",
    "salary": "회사내규에 따름 - 면접 후 결정",
    "employmentType": "정규직, 계약직, 아르바이트",
    "createdAt": "2025-07-15T11:46:00",
    "combinedContent": "경력 학력무관 간호조무사 없음 회사내규에 따름 회사내규에 따름 - 면접 후 결정 정규직, 계약직, 아르바이트",
    "jobKeywords": "분당 도보 간호사 거리 서현역 응급구조사 간호조무사",
    "scrapCount": 0
  }
}
```

4. 프로젝트 수행 경과

채용공고 수집과 Elasticsearch 검색 구현 1

크롤링 → 전처리 → DB 저장 → Elasticsearch 색인 → **검색 API 구현**

✓ 결과 예시 : 채용 공고 검색

개발자

공고내용

서울

회사명

시작일

2025-07-11

마감일

2025-07-25

검색

전체

miridiH

[미리캔버스] 프론트엔드 개발자

마감일: 25. 7. 11.
스크랩 수: 0

miridiH

[미리캔버스] 시니어 프론트엔드 개발자

마감일: 25. 7. 25.
스크랩 수: 0

miridiH

[미리캔버스] 시니어 백엔드 개발자

마감일: 25. 7. 16.
스크랩 수: 0

ezCure

병원정보시스템 S/W 경력 개발자

마감일: 25. 7. 15.
스크랩 수: 0

Omi KOREA

신입] ASP.NET개발자

마감일: 25. 7. 20.
스크랩 수: 0

React Native

React Native 리드 개발자 (5년차 이상)

마감일: 25. 7. 25.
스크랩 수: 0

Omi KOREA

ASP.NET 중급개발자

마감일: 25. 7. 20.
스크랩 수: 0

백엔드 개발자 (1년 이상)

마감일: 25. 7. 21.
스크랩 수: 0

<검색 +정렬 구현 부분>

- 채용공고 : 공고이름, 공고내용, 지역, 회사명, 시작일, 마감일
- 관리자-회원관리 : 이메일, 닉네임
- 관리자-공고관리 : 공고이름, 공고내용, 지역, 회사명, 시작일, 마감일
- 관리자-지원관리 : 공고이름, 회사명, 이메일, 시작일, 마감일

4. 프로젝트 수행 경과

User, ApplyHistory Elasticsearch 검색 구현 2

DB 저장/Elasticsearch 색인 → 검색 API 구현

✓ DB 저장/Elasticsearch 색인

```
public Users saveUser(Users user) { 1 usage 👤 green414
    validateDuplicateUser(user);

    Users savedUser = userRepository.save(user);

    UserDocument userDoc = toUserDocument(savedUser);
    userDocumentRepository.save(userDoc);

    return savedUser;
}
```

User와 ApplyHistory의 데이터 저장 및 검색 방식은 동일

```
// 회원가입 폼
@GetMapping("/signup") no usages 👤 tjoeunJava
public String userForm(Model model) {
    model.addAttribute(attributeName: "userFormDto", new UserFormDto());
    return "user/signup";
}

// 회원가입 처리
@PostMapping("/signup") no usages 👤 tjoeunJava
public String newUser(@Valid @ModelAttribute("userFormDto") UserFormDto dto,
    BindingResult bindingResult,
    Model model) {

    if (bindingResult.hasErrors()) {
        return "user/signup";
    }
    try {
        Users user = Users.createUser(dto, passwordEncoder);
        userService.saveUser(user);
    } catch (IllegalStateException e) {
        model.addAttribute(attributeName: "errorMessage", e.getMessage());
        return "user/signup";
    }
    return "redirect:/";
}
```


4. 프로젝트 수행 경과

User, ApplyHistory Elasticsearch 검색 구현 2

DB 저장/Elasticsearch 색인 → **검색 API 구현**

✓ 검색 API 구현

UserSearchService.searchUsers – 검색 쿼리 로직

```
public Page<UserListDto> searchUsers( 1 usage 2 green4141 +2
    String email, String nickname, String sortBy, int page, int size
) throws IOException {

    int from = (page - 1) * size;

    // 필터 조건 설정
    List<Query> mustQueries = new ArrayList<>();

    if (email != null && !email.isBlank()) {
        String escapedEmail = escapeWildcard(email);
        mustQueries.add(Query.of( Builder q -> q
            .wildcard( Builder w -> w
                .field( value: "userEmail")
                .wildcard( value: "*" + escapedEmail + "*" )
            )
        ));
    }

    if (nickname != null && !nickname.isBlank()) {
        String escapedNickname = escapeWildcard(nickname);
        mustQueries.add(Query.of( Builder q -> q
            .wildcard( Builder w -> w
                .field( value: "userNickname")
                .wildcard( value: "*" + escapedNickname + "*" )
            )
        ));
    }

    // 정렬 조건
    SortOptions sort = switch (sortBy) {
        case "email" -> SortOptions.of( Builder s -> s.field( Builder f -> f.field( value: "userEmail").order(SortOrder.Asc)));
        case "emailDesc" -> SortOptions.of( Builder s -> s.field( Builder f -> f.field( value: "userEmail").order(SortOrder.Desc)));
        case "nickname" -> SortOptions.of( Builder s -> s.field( Builder f -> f.field( value: "userNickname").order(SortOrder.Asc)));
        case "nicknameDesc" -> SortOptions.of( Builder s -> s.field( Builder f -> f.field( value: "userNickname").order(SortOrder.Desc)));
        case "oldest" -> SortOptions.of( Builder s -> s.field( Builder f -> f.field( value: "userIdx").order(SortOrder.Asc)));
        default -> SortOptions.of( Builder s -> s.field( Builder f -> f.field( value: "userIdx").order(SortOrder.Desc)));
    };

    SearchResponse<UserDocument> response = esClient.search( Builder s -> s
        .index(indexName)
        .query( Builder q -> q
            .bool( Builder b -> b.must(mustQueries))
        )
        .sort(sort)
        .from(from)
        .size(size),
        UserDocument.class
    );

    List<UserListDto> content = response.hits().hits().stream() Stream<Hit<...>>
        .map(Hit::source) Stream<UserDocument>
        .map( UserDocument doc -> UserListDto.builder()
            .userIdx(doc.getUserIdx())
            .userName(doc.getUserName())
            .userEmail(doc.getUserEmail())
            .userNickname(doc.getUserNickname())
            .userBirth(Instant.ofEpochMilli(doc.getUserBirth()) Instant
                .atZone(ZoneId.systemDefault()) ZonedDateTime
                .toLocalDate())
            .build() Stream<UserListDto>
        .toList();

    return new PageImpl<>(content, PageRequest.of( pageNumber: page - 1, size), response.hits().total().value());
}
```

```
// 검색 요청
SearchResponse<UserDocument> response = esClient.search( Builder s -> s
    .index(indexName)
    .query( Builder q -> q
        .bool( Builder b -> b.must(mustQueries))
    )
    .sort(sort)
    .from(from)
    .size(size),
    UserDocument.class
);

List<UserListDto> content = response.hits().hits().stream() Stream<Hit<...>>
    .map(Hit::source) Stream<UserDocument>
    .map( UserDocument doc -> UserListDto.builder()
        .userIdx(doc.getUserIdx())
        .userName(doc.getUserName())
        .userEmail(doc.getUserEmail())
        .userNickname(doc.getUserNickname())
        .userBirth(Instant.ofEpochMilli(doc.getUserBirth()) Instant
            .atZone(ZoneId.systemDefault()) ZonedDateTime
            .toLocalDate())
        .build() Stream<UserListDto>
    .toList();

return new PageImpl<>(content, PageRequest.of( pageNumber: page - 1, size), response.hits().total().value());
}
```

4. 프로젝트 수행 경과

User, ApplyHistory Elasticsearch 검색 구현 2
DB 저장/Elasticsearch 색인 → **검색 API 구현**

✓ 결과 화면

회원관리

검색

가입일 : 최신순 ▼

fffff@ddddd
ssssss

삭제

lll@naver.com
관리자22

삭제

navmde@naver.com
이름이

삭제

ooo@naver.com
오오오

삭제

ananan@naver.com
애애펑4

삭제

지원관리

시작일
연도-월-일

마감일
연도-월-일

검색

상태 전체 ▼

전체 ▼

[기숙사/식사/셔틀제공] 떼제베CC-식음료(홀서버) 모집 (주5일 근무)
올데이 청주떼제베
user@naver.com | 사용자

합격

데브옵스 엔지니어
키즈노트
user@naver.com | 사용자

진행중

[오브제 성형외과] 간호팀 수술실 어시 팀원 모집합니다.
오브제성형외과의원
user@naver.com | 사용자

진행중

4. 프로젝트 수행 경과

대시보드

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 분석기 설정(settings)

1. korean_custom 분석기 생성
2. nori_tokenizer 분석기 생성
3. 불용어 필터(my_stop) 적용

✓ 필드 매핑 설정(mappings)

jobKeywords, title, combinedContent -> 분석기 적용

company, location -> 정확 매칭용 keyword 타입

createdAt, deadline -> 날짜 타입

나머지 필드들 → text 타입

```
.index( value: "recruitments")
.settings( Builder s -> s.withJson(new StringReader( s: ""
{
  "analysis": {
    "analyzer": {
      "korean_custom": {
        "type": "custom",
        "tokenizer": "nori_tokenizer",
        "filter": ["lowercase", "nori_readingform", "my_stop"]
      }
    },
    "filter": {
      "my_stop": {
        "type": "stop",
        "stopwords": ["ㄴ", "이", "자", "직", "주요", "사", "관리", "동", "님", "만수", "통하", "통한", "통해"]
      }
    }
  }
}
"")))
```

```
.mappings( Builder mb -> mb
    .properties( key: "jobKeywords", Builder p -> p.text( Builder t -> t.analyzer( value: "korean_custom").fielddata( value: true)))
    .properties( key: "title", Builder p -> p.text( Builder t -> t.analyzer( value: "korean_custom")))
    .properties( key: "combinedContent", Builder p -> p.text( Builder t -> t.analyzer( value: "korean_custom")))
    .properties( key: "company", Builder p -> p.keyword( Builder k -> k))
    .properties( key: "deadline", Builder p -> p.date( Builder d -> d.format( value: "yyyy-MM-dd")))
    .properties( key: "createdAt", Builder p -> p.date( Builder d -> d.format( value: "yyyy-MM-dd")))
    .properties( key: "location", Builder p -> p.keyword( Builder k -> k))
    .properties( key: "_class", Builder p -> p.keyword( Builder k -> k.index( value: false).docValues(false)))
)
```

28

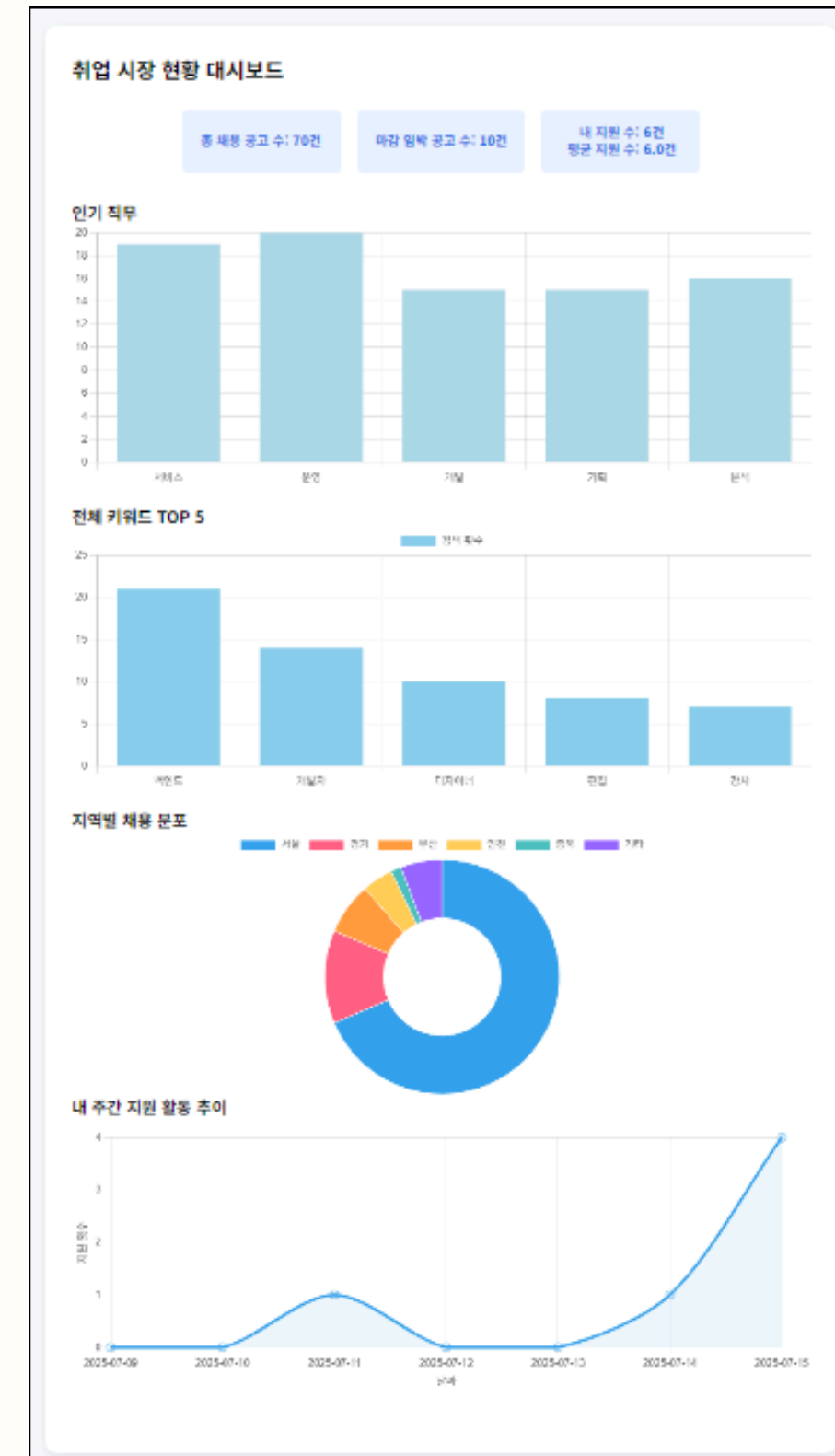
4. 프로젝트 수행 경과

대시보드

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 사용자 대시보드 시각화 요소 구성

시각화 영역	형태	데이터 설명
요약 통계 카드	숫자 카드	총 채용 공고 수 / 지원 수 마감 임박 공고 수
인기직무	Bar Chart	jobKeywords.keyword 기반 top 5
전체 키워드 TOP 5	Bar Chart	combinedContent에서 추출된 키워드 집계
지역별 채용 분포	Donut Chart	location.keyword 기반 채용 비율
내 주간 지원 활동 추이	Line Chart	최근 7일 ApplyHistory.createdAt 추이



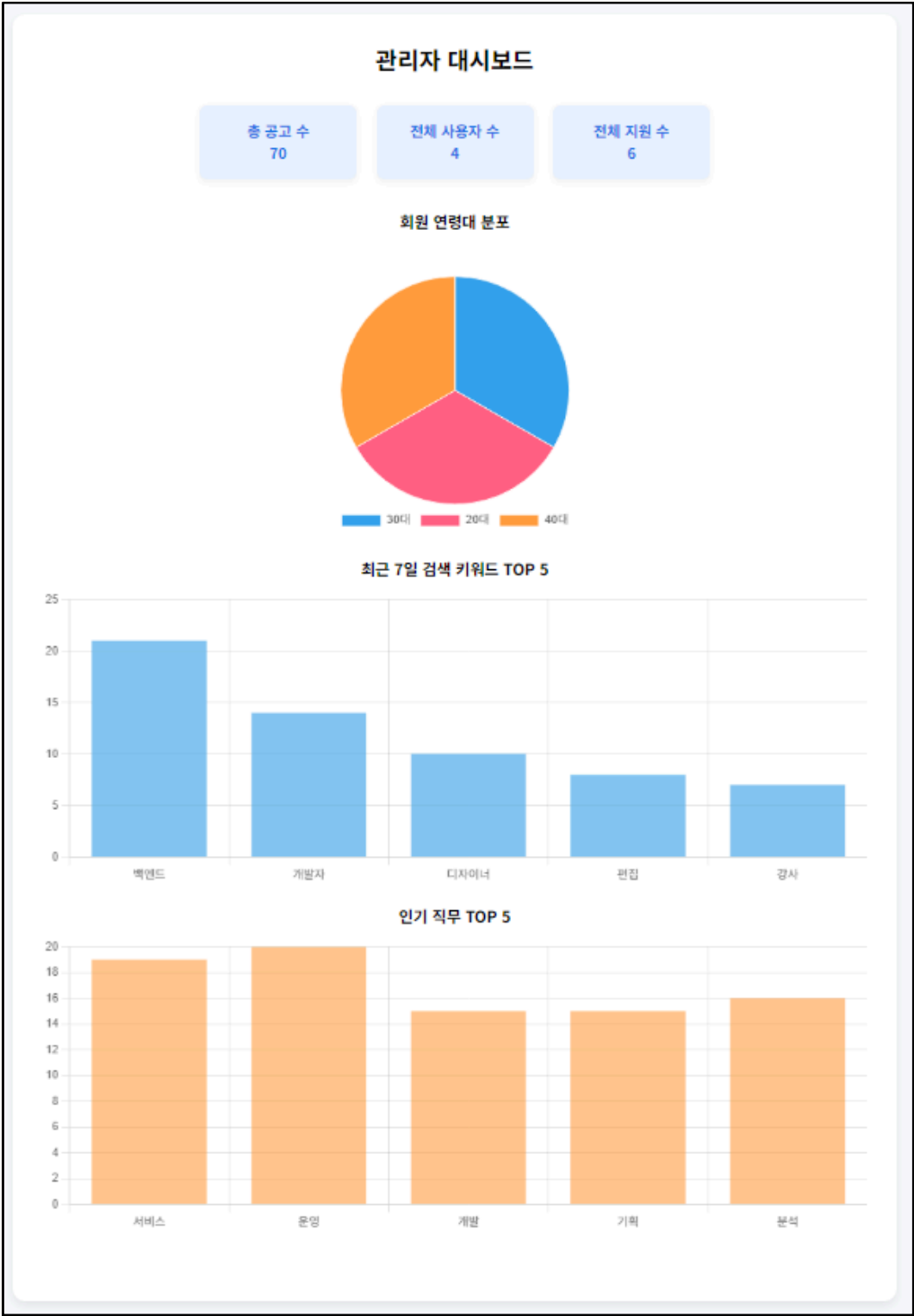
4. 프로젝트 수행 경과

대시보드

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 관리자 대시보드 시각화 요소 구성

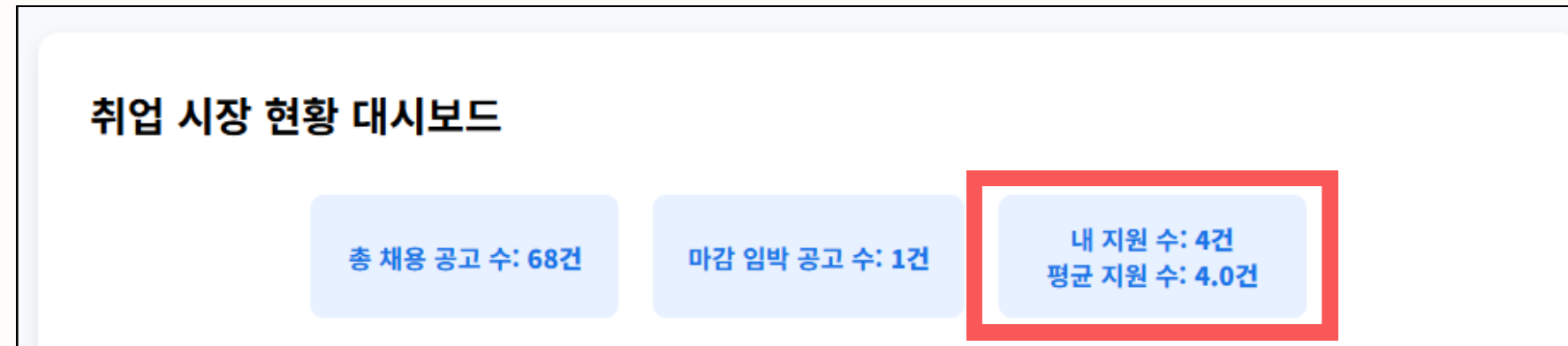
시각화 영역	형태	데이터 설명
요약 통계 카드	숫자 카드	총 공고 수 / 전체 사용자 수 / 전체 지원 수
회원 연령대 분포	Pie Chart	사용자 나이 기준으로 비율 표시
최근 7일 검색 키워드 TOP 5	Bar Chart	사용자 검색 로그에서 searchKeyword 집계
인기 직무 TOP 5	Bar Chart	jobKeywords.keyword 기반 top 5



4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)



✓ 내 지원 수

```
public DashboardDTO getUserDashboardData(Long userIdx) {  
    int myApplyCount = applyHistoryRepository.countByUserIdx(userIdx);  
}
```

```
@Query("SELECT COUNT(a) FROM ApplyHistory a WHERE a.user.userIdx = :userIdx")  
int countByUserIdx(@Param("userIdx") Long userIdx);
```

SQL 기반 로그인된 사용자 기준으로 지원 이력 수 조회

✓ 평균 지원 수

```
List<Long> counts = applyHistoryRepository.findApplyCountsPerUser();  
double avgApplyCount = counts.stream().mapToLong(Long::longValue).average().orElse(0.0);
```

```
@Query("SELECT COUNT(h) FROM ApplyHistory h GROUP BY h.user.userIdx")  
List<Long> findApplyCountsPerUser();
```

SQL 기반 사용자별 지원 건수를 조회

counts 리스트 Stream 반환 → mapToLong을 통해 long stream 으로 전환 → .average() 평균 계산 → 만약 리스트가 비어 있으면 0.0 반환

4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

취업 시장 현황 대시보드

총 채용 공고 수: 68건

마감 임박 공고 수: 1건

내 지원 수: 4건
평균 지원 수: 4.0건

✓ 총 채용 공고 수

```
public long countRecruitments() {  
    try {  
        CountRequest request = CountRequest.of( Builder c -> c.index( value: "recruitments" ));  
        CountResponse response = elasticSearchClient.count(request);  
        return response.count();  
    } catch (IOException e) {  
        e.printStackTrace();  
        return 0;  
    }  
}
```

recruitments 인덱스 전체 문서 수를 세는 쿼리

```
public long countClosingSoonRecruitments() {  
    try {  
        String today = LocalDate.now().toString();  
        String soon = LocalDate.now().plusDays( daysToAdd: 7 ).toString();  
  
        System.out.println(" 오늘 날짜: " + today);  
        System.out.println(" 마감 기준 날짜: " + soon);  
  
        CountRequest request = CountRequest.of( Builder c ->  
            c.index( value: "recruitments" )  
                .query( Builder q -> q.range( Builder r -> r  
                    .field( value: "deadline" )  
                    .gte(JsonData.of(today))  
                    .lte(JsonData.of(soon))  
                ))  
        );  
  
        long count = elasticSearchClient.count(request).count();  
        System.out.println(" 마감 임박 공고 수: " + count);  
  
        return count;  
    } catch (IOException e) {  
        e.printStackTrace();  
        return 0;  
    }  
}
```

✓ 마감 임박 공고 수

- 날짜 조건 필터 검색

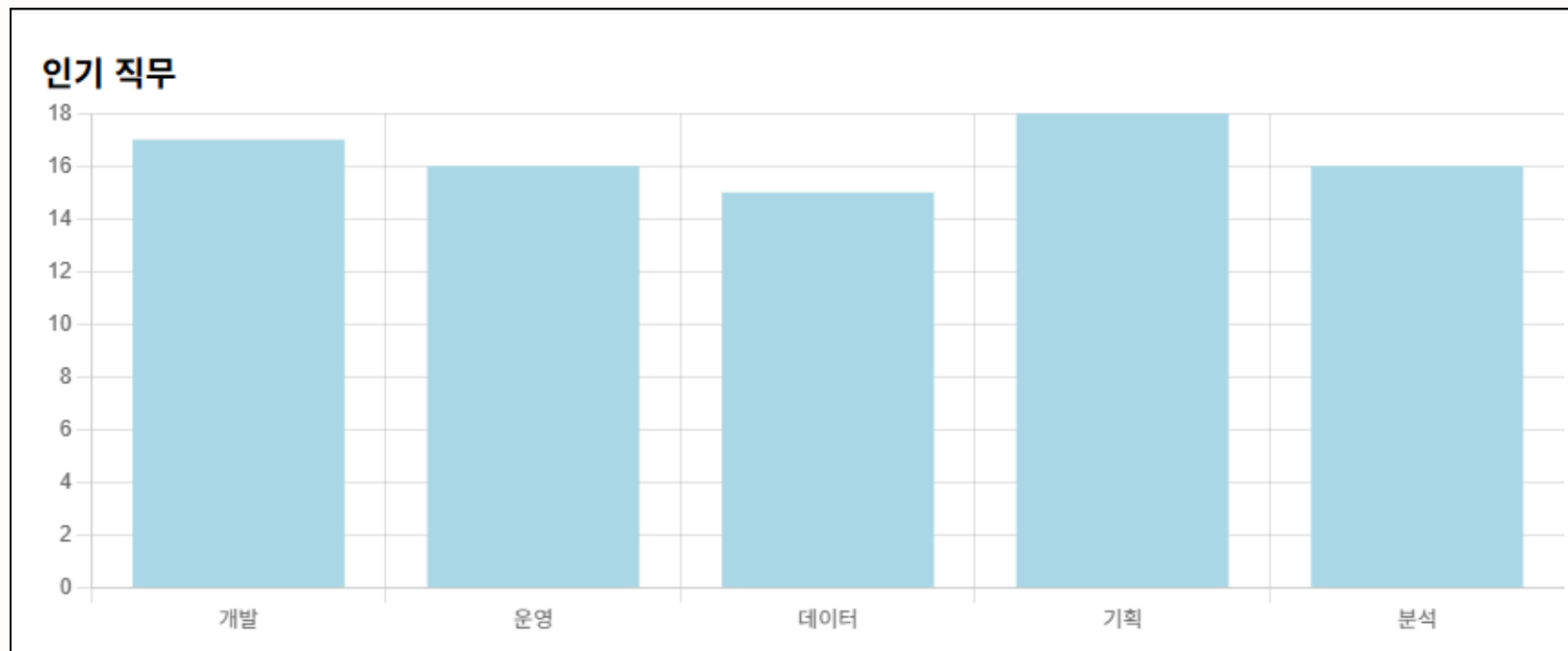
- deadline 필드를 기준으로
오늘로부터 7일 이내에 마감되는
채용공고 카운트
- Elasticsearch의
count API + range query 사용

4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 인기 직무 TOP 5 - SPRING



search() 메소드 사용

→ Elasticsearch 에 검색 쿼리 전송 (통계 결과만 필요하기 때문에

aggregation size = 0)

→ recruitments 인덱스의 jobKeywords 필드 대상으로

가장 자주 등장하는 값 집계 (fielddata: true => 집계 허용)

→ 집계 결과를 bucket 리스트 형태로 저장

→ 각 버킷을 키워드명, 등장 횟수 형태의 Map 으로 변환

```
public Map<String, Long> getTopRoles(int size) {
    try {
        SearchResponse<Void> response = elasticsearchClient.search( Builder s -> s
            .index( value: "recruitments")
            .size( value: 0)
            .aggregations( key: "top_roles", Builder a -> a
                .terms( Builder t -> t
                    .field( value: "jobKeywords")
                    .size(size)
                )
            ), Void.class);

        return response.aggregations().get("top_roles").sterms().buckets().array().stream()
            .collect(Collectors.toMap(
                StringTermsBucket b -> b.key().stringValue(),
                StringTermsBucket b -> b.docCount()
            ));
    } catch (IOException e) {
        e.printStackTrace();
        return Map.of();
    }
}
```


4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 인기 직무 TOP 5 - 시각화(Chart.js)

💡 Chart.js 란?

JavaScript 기반 차트 라이브러리

bar, line, pie, doughnut, radar 등 다양한 차트 타입 지원

Canvas 기반으로 차트 렌더링 → 간단한 문법으로 시각화 가능

```
<div class="chart-container">
  <h3>인기 직무</h3>
  <canvas id="popularRoleChart"></canvas>
</div>
```

실제 Chart.js 가 그려질 캔버스 영역 지정

```
const popularRoleLabels = /*[`${dashboard.popularRoles}`]*/ [];
const popularRoleCounts = /*[`${dashboard.popularRoleCounts}`]*/ [];

new Chart(document.getElementById('popularRoleChart'), {
  type: 'bar',
  data: {
    labels: popularRoleLabels,
    datasets: [{
      label: '광고 수',
      data: popularRoleCounts,
      backgroundColor: 'lightblue'
    }]
  },
  options: {
    plugins: {
      legend: {
        display: false
      }
    }
  }
});
```



SPRING → JS 인기 직무 전달

type : 바 차트로 생성

labels: popularRoleLabels : X축 라벨 (직무명)

data: popularRoleCounts : Y축 값 (해당 직무 광고 수)

backgroundColor : 막대 색상 설정

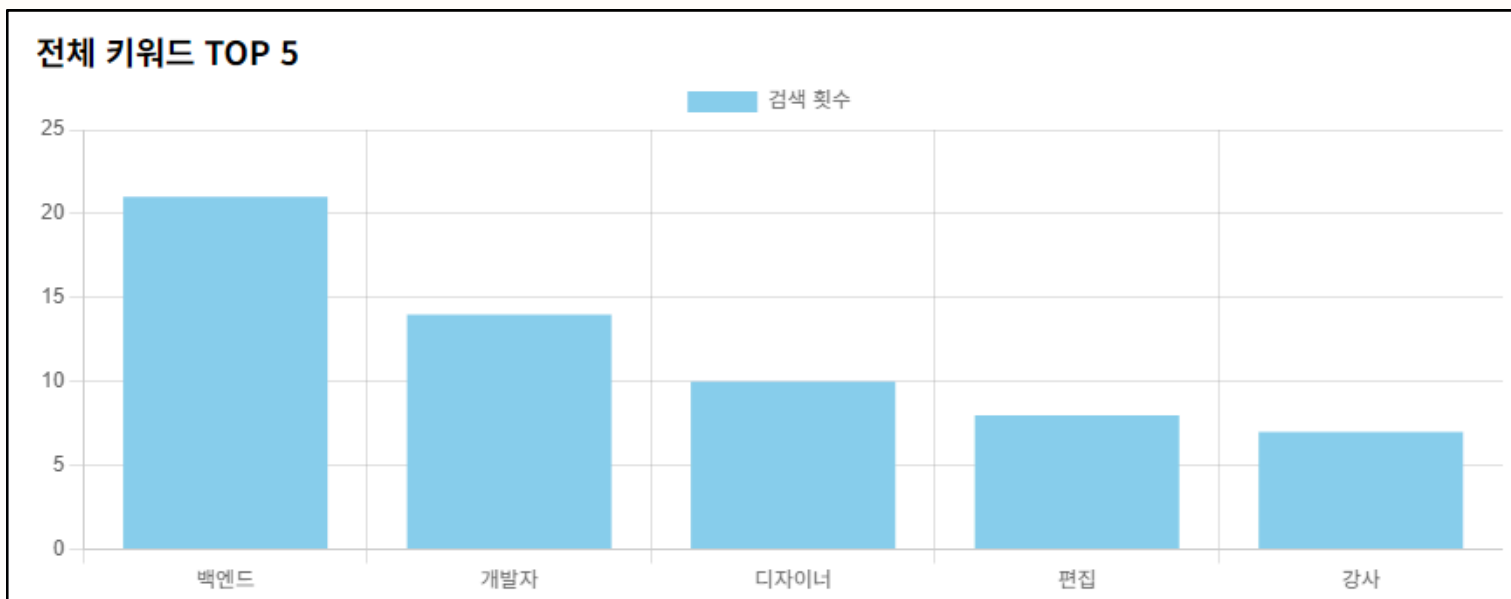
legend.display: false : 범례(legend) 안 보이도록 설정

4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 전체 키워드 검색 TOP 5 - SPRING



SearchLogRepository

```
@Query("SELECT s.keyword, COUNT(s) FROM SearchLog s GROUP BY s.keyword ORDER BY COUNT(s) DESC")  
List<Object[]> findTopKeywords(Pageable pageable);
```

1. SearchLog 엔티티

- keyword 컬럼 기준으로 해당 키워드 검색 횟수 카운트 후 많은 순으로 반환
- 반환 타입 : List<Object[]> => Object[0] - 키워드 / Object[1] - 횟수

DashboardService

```
public List<Map.Entry<String, Long>> getTopSearchedKeywords(int size) {  
    List<Object[]> result = searchLogRepository.findTopKeywords(PageRequest.of(0, size));  
    return result.stream()  
        .map(row -> Map.entry((String) row[0], ((Number) row[1]).longValue()))  
        .collect(Collectors.toList());  
}
```

2. getTopSearchedKeywords() 메소드

- Object[] 리스트 받음 → Map 구조 반환

4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 인기 직무 TOP 5 - 시각화(Chart.js)

```
<div class="chart-container">
  <h3>전체 키워드 TOP 5</h3>
  <canvas id="keywordChart"></canvas>
</div>
```

실제 Chart.js 가 그려질 캔버스 영역 지정

```
const topKeywordLabels = /*[[${dashboard.topKeywordLabels}]]*/ [];
const topKeywordCounts = /*[[${dashboard.topKeywordCounts}]]*/ [];

new Chart(document.getElementById('keywordChart'), {
  type: 'bar',
  data: {
    labels: topKeywordLabels,
    datasets: [{
      label: '검색 횟수',
      data: topKeywordCounts,
      backgroundColor: 'skyblue'
    }]
  },
  options: {
    plugins: {}
  }
});
```



SPRING → JS 인기 키워드 전달

type : 바 차트로 생성

labels: topKeywordLabels : X축 라벨 (키워드명)

data: topKeywordCounts : Y축 값 (키워드 검색 횟수)

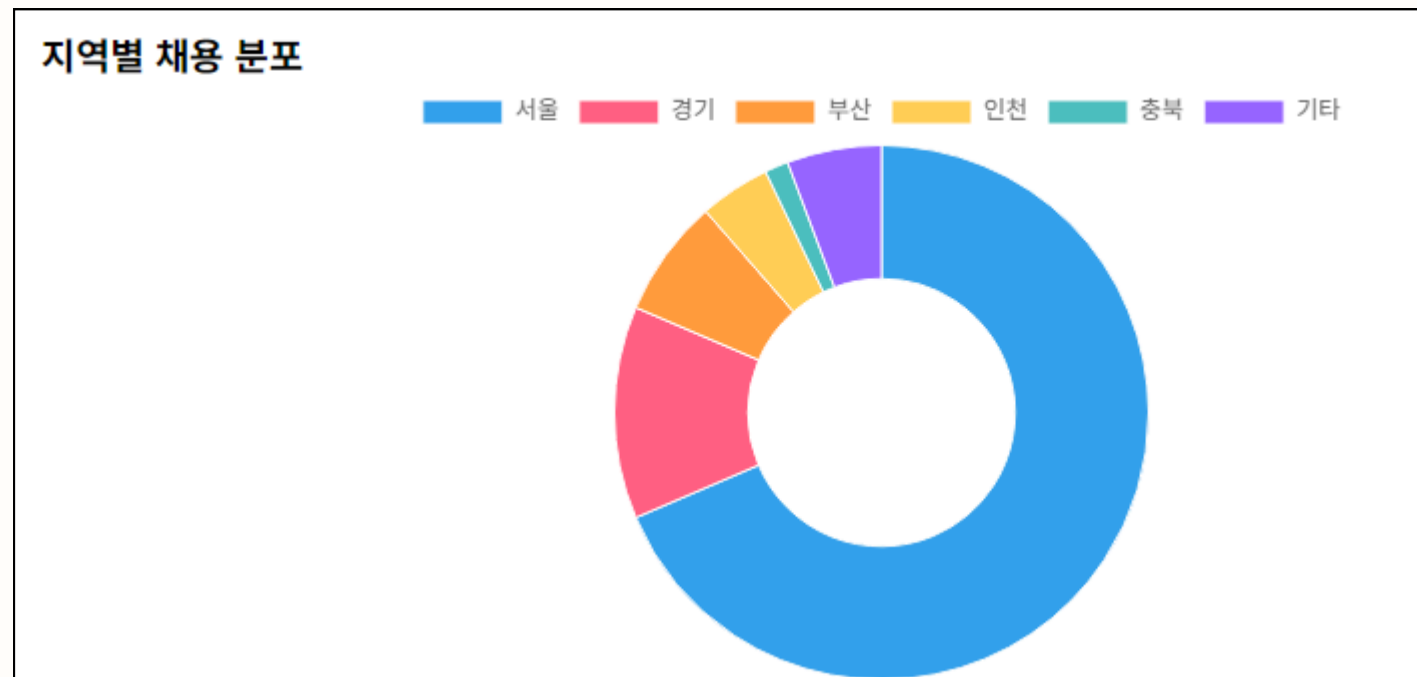
backgroundColor : 막대 색상 설정

4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 전체 키워드 검색 TOP 5 - SPRING



1. Elasticsearch 에서 location 필드만 가져옴
 - recruitments 인덱스에서 최대 1000개의 문서 중 location 필드가 있는 문서 대상으로
 - location 만 가져옴

ElasticsearchService

```
// 지역별 채용 분포
public Map<String, Long> getRegionDistribution() { 1 usage  bhjava
    try {
        SearchResponse<Map> response = elasticSearchClient.search( Builder s -> s
            .index( value: "recruitments")
            .size( value: 1000)
            .query( Builder q -> q.exists( Builder e -> e.field( value: "location")))
            .source( Builder src -> src.filter( Builder f -> f.includes( value: "location"))),
            Map.class
        );

        Map<String, Long> grouped = response.hits().hits().stream() Stream<Hit<...>>
            .map( Hit<Map> hit -> {
                Map<String, Object> source = hit.source();
                String raw = (String) source.get("location");
                if (raw == null) return "기타";

                if (raw.startsWith("서울")) return "서울";
                if (raw.startsWith("경기")) return "경기";
                if (raw.startsWith("대전")) return "대전";
                if (raw.startsWith("부산")) return "부산";
                if (raw.startsWith("인천")) return "인천";
                if (raw.startsWith("광주")) return "광주";
                if (raw.startsWith("대구")) return "대구";
                if (raw.startsWith("울산")) return "울산";
                if (raw.startsWith("세종")) return "세종";
                if (raw.startsWith("전북") || raw.startsWith("전라북도")) return "전북";
                if (raw.startsWith("전남") || raw.startsWith("전라남도")) return "전남";
                if (raw.startsWith("충북") || raw.startsWith("충청북도")) return "충북";
                if (raw.startsWith("충남") || raw.startsWith("충청남도")) return "충남";
                if (raw.startsWith("경북") || raw.startsWith("경상북도")) return "경북";
                if (raw.startsWith("경남") || raw.startsWith("경상남도")) return "경남";
                if (raw.startsWith("강원")) return "강원";
                if (raw.startsWith("제주")) return "제주";
                return "기타";
            }) Stream<String>
            .collect(Collectors.groupingBy( String r -> r, Collectors.counting()));
    }
}
```

1. 지역명 그룹핑 처리

4. 프로젝트 수행 경과

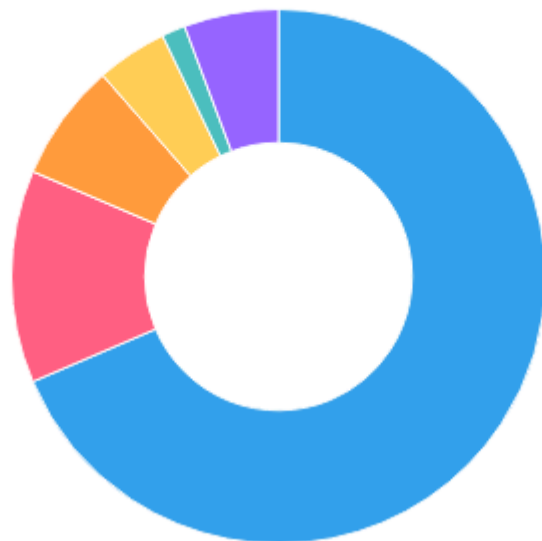
대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 전체 키워드 검색 TOP 5 - SPRING

지역별 채용 분포

서울 경기 부산 인천 충북 기타



2. 상위 5개 지역은 그대로 두고,
나머지는 전부 '기타' 처리

DashboardService

```
Map<String, Long> regionData = elasticsearchService.getRegionDistribution();
```

3. getRegionDistribution() 메서드를 통해 데이터를 받아오고,
이를 프론트에서 Chart.js 도넛 차트 형태로 시각화

ElasticsearchService

```
Map<String, Long> sorted = grouped.entrySet().stream()
    .sorted(Map.Entry.comparingByValue().reversed())
    .collect(Collectors.toList())
    .stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        List<Entry<String, Long>> list -> {
            Map<String, Long> top5 = new LinkedHashMap<>();
            long etcSum = 0;
            for (int i = 0; i < list.size(); i++) {
                Map.Entry<String, Long> entry = list.get(i);
                if (i < 5) {
                    top5.put(entry.getKey(), entry.getValue());
                } else {
                    etcSum += entry.getValue();
                }
            }
            if (etcSum > 0) {
                top5.put("기타", etcSum);
            }
            return top5;
        }
    ));
```


4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 전체 키워드 검색 TOP 5 - 시각화(Chart.js)

```
<div class="chart-container">
  <h3>지역별 채용 분포</h3>
  <canvas id="regionChart"></canvas>
</div>
```

실제 Chart.js 가 그려질 캔버스 영역 지정

```
const regionLabels = /*[[${dashboard.regionLabels}]]*/ [];
const regionCounts = /*[[${dashboard.regionCounts}]]*/ [];

const ctx = document.getElementById('regionChart').getContext('2d');

new Chart(ctx, {
  type: 'doughnut',
  data: {
    labels: regionLabels,
    datasets: [{
      data: regionCounts,
      borderWidth: 1
    }]
  }
});
```

SPRING → JS 지역명 전달

차트를 그릴 캔버스의 2D 그래픽 컨텍스트 지정

type : 도넛형 차트로 생성

labels: regionLabels: X축 라벨 (지역명)

data: regionCounts: Y축 값 (지역 카운트 수)

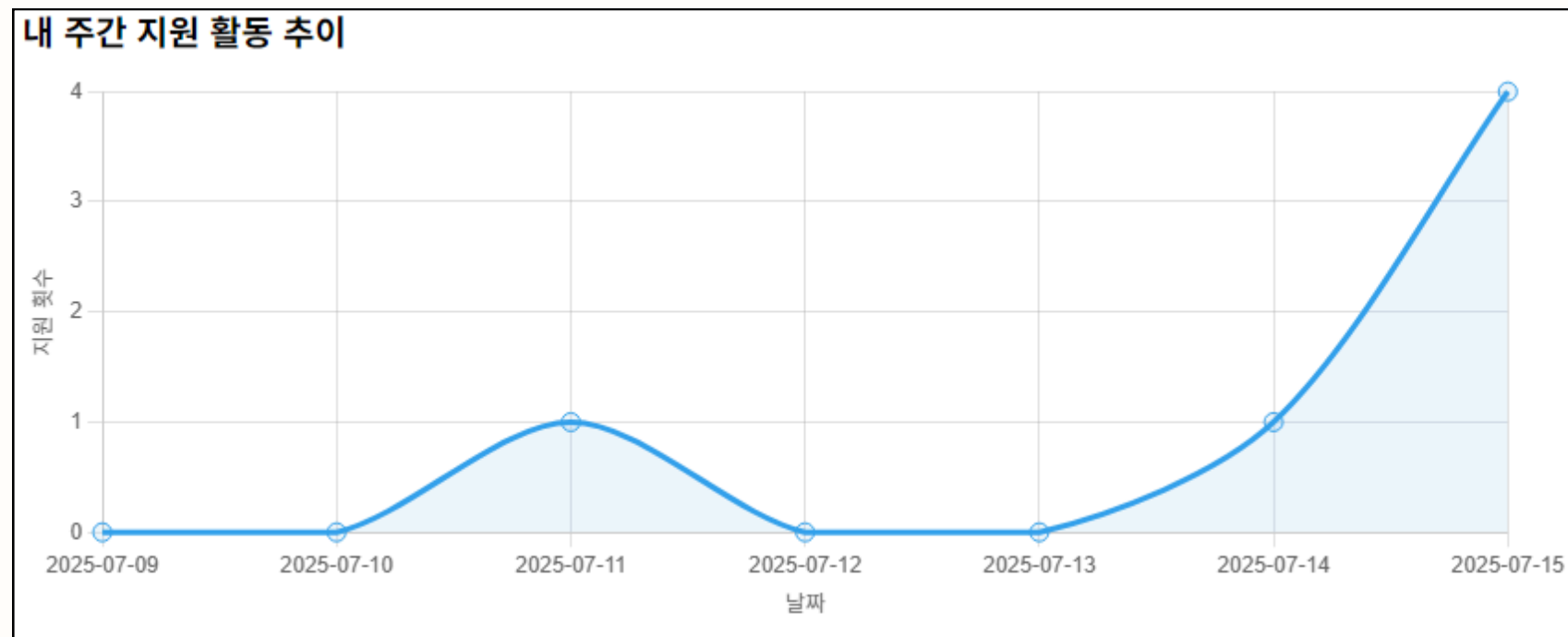
borderWidth: 도넛 경계선 두께

4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 전체 키워드 검색 TOP 5 - SPRING



ElasticsearchService

```
public Map<String, Long> getUserWeeklyApplyTrend(Long userIdx, ApplyHistoryRepository applyHistoryRepository) {
    LocalDate today = LocalDate.now();
    LocalDate weekAgo = today.minusDays(daysToSubtract: 6);

    List<Object[]> result = applyHistoryRepository.countDailyApplies(userIdx, weekAgo, today);

    Map<String, Long> trend = new LinkedHashMap<>();
    for (int i = 0; i < 7; i++) {
        String date = weekAgo.plusDays(i).toString();
        trend.put(date, 0L);
    }

    for (Object[] row : result) {
        String date = row[0].toString();
        Long count = ((Number) row[1]).longValue();
        trend.put(date, count);
    }

    return trend;
}
```

DashboardService

```
Map<String, Long> applyTrend = elasticSearchService.getUserWeeklyApplyTrend(userIdx, applyHistoryRepository);
List<String> applyTrendLabels = new ArrayList<>(applyTrend.keySet());
List<Long> applyTrendCounts = new ArrayList<>(applyTrend.values());
```

날짜 라벨, 지원 횟수 리스트를 분리하여 프론트에 넘김

weekAgo 부터 today 까지 7일간의 날짜 문자열을 키로 하는 Map 생성
DB에서 사용자의 지원 내역을 날짜별로 그룹화하여 카운트
날짜별 count 값을 trend에 채움 (없는 날짜는 0 유지)

4. 프로젝트 수행 경과

대시보드 - 사용자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 전체 키워드 검색 TOP 5 - 시각화(Chart.js)

```
<div class="chart-container">
  <h3>지역별 채용 분포</h3>
  <canvas id="regionChart"></canvas>
</div>
```

실제 Chart.js 가 그려질 캔버스 영역 지정

```
const applyTrendLabels = /*[[$dashboard.applyTrendLabels]]*/ [];
const applyTrendCounts = /*[[$dashboard.applyTrendCounts]]*/ [];

const applyTrendCtx = document.getElementById('applyTrendChart');
if (applyTrendCtx) {
  new Chart(applyTrendCtx.getContext('2d'), {
    type: 'line',
    data: {
      labels: applyTrendLabels,
      datasets: [{
        data: applyTrendCounts,
        borderColor: 'rgba(54, 162, 235, 1)',
        backgroundColor: 'rgba(54, 162, 235, 0.1)',
        tension: 0.3,
        fill: true,
        pointRadius: 5,
        pointHoverRadius: 7,
      }]
    },
    options: {
      responsive: true,
      scales: {
        y: {
          beginAtZero: true,
          ticks: {
            stepSize: 1
          },
          title: {
            display: true,
            text: '지원 횟수'
          }
        }
      }
    }
  });
}
```

```
},
x: {
  title: {
    display: true,
    text: '날짜'
  }
},
plugins: {
  legend: {
    display: false
  },
  tooltip: {
    callbacks: {
      label: function(context) {
        return `${context.parsed.y}건 지원`;
      }
    }
  }
}
});
} else {
  console.error("🚫 applyTrendChart 요소를 찾을 수 없습니다.");
}
```

마우스 올렸을 시 나오는 말풍선 설정

type : 선 그래프 지정

labels: applyTrendLabels: X축 라벨 (날짜)

data: applyTrendCounts: Y축 값 (지원 수)

datasets: 그래프의 시각적 설정

4. 프로젝트 수행 경과

대시보드 - 관리자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)



✓ 총 공고 수, 전체 사용자 수, 전체 지원 수

DashboardService

```
// 전체 지원 수
long totalApplyCount = applyHistoryRepository.count();

// 전체 사용자 수
long totalUserCount = userRepository.count();

// 전체 공고 수
long totalJobCount = elasticsearchService.countRecruitments();
```

총 공고 수 재사용

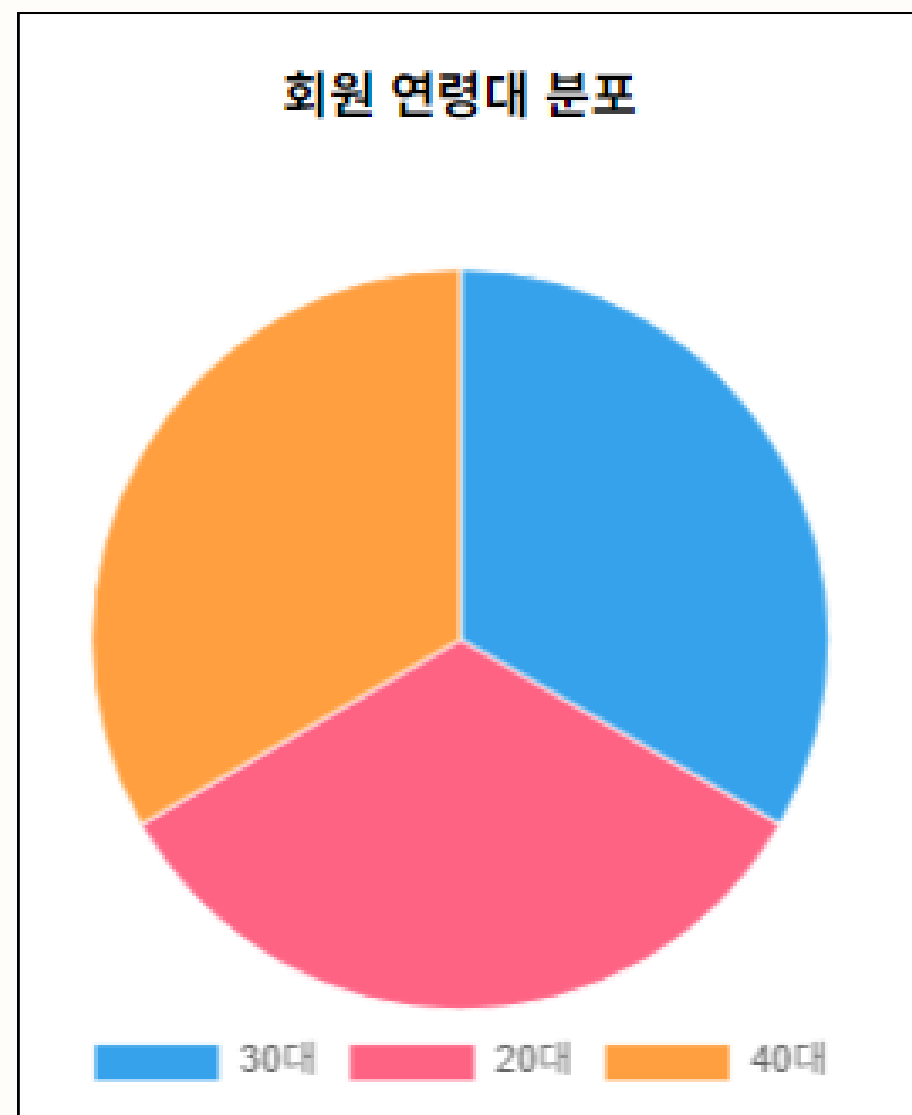
```
<div class="metric-boxes">
  <div class="metric">
    <div>총 공고 수</div>
    <div th:text="${dashboard.totalJobCount}">0</div>
  </div>
  <div class="metric">
    <div>전체 사용자 수</div>
    <div th:text="${dashboard.totalUserCount}">0</div>
  </div>
  <div class="metric">
    <div>전체 지원 수</div>
    <div th:text="${dashboard.totalApplyCount}">0</div>
  </div>
</div>
```

4. 프로젝트 수행 경과

대시보드 - 관리자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 회원 연령대 분포 - SPRING



DashboardService

```
Map<String, Long> ageGroups = userRepository.findAll().stream() Stream<Users>
    .filter( Users user -> user.getUserRole() == UserRole.USER)
    .map( Users user -> {
        int age = LocalDate.now().getYear() - user.getUserBirth().getYear();
        if (age < 20) return "10대 이하";
        else if (age < 30) return "20대";
        else if (age < 40) return "30대";
        else if (age < 50) return "40대";
        else if (age < 60) return "50대";
        else return "60대 이상";
    }) Stream<String>
    .collect(Collectors.groupingBy( String ageGroup -> ageGroup, Collectors.counting()));

List<String> ageGroupLabels = new ArrayList<>(ageGroups.keySet());
List<Long> ageGroupCounts = new ArrayList<>(ageGroups.values());
```

- UserRepository에서 모든 유저 불러온 후 일반 사용자(USER)만 필터링
- user.getUserBirth() → 나이 계산 후 연령대 분류
- 프론트에 전달할 수 있도록 리스트화

4. 프로젝트 수행 경과

대시보드 - 관리자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

✓ 회원 연령대 분포 - 시각화(Chart.js)

DashboardService

```
<div class="chart-container">
  <h3>회원 연령대 분포</h3>
  <canvas id="ageGroupChart" width="400" height="400"></canvas>
</div>
```

- type : pie 그래프 지정
- labels: ageGroupLabels: X축 라벨 (연령대)
- data: ageGroupCounts: Y축 값 (회원수)
- options: 그래프의 시각적 설정

```
const ageGroupLabels = [[${dashboard.ageGroupLabels}]];
const ageGroupCounts = [[${dashboard.ageGroupCounts}]];

const ctx = document.getElementById('ageGroupChart').getContext('2d');
new Chart(ctx, {
  type: 'pie',
  data: {
    labels: ageGroupLabels,
    datasets: [{
      label: '회원 수',
      data: ageGroupCounts,
      borderWidth: 1
    }]
  },
  options: {
    responsive: true,
    plugins: {
      legend: {
        position: 'bottom'
      },
      title: {
        display: true,
      }
    }
  }
})
```

4. 프로젝트 수행 경과

대시보드 - 관리자

Kibana에서 인덱스 패턴 설정 → 시각화 구성 → 대시보드 구성(사용자/관리자)

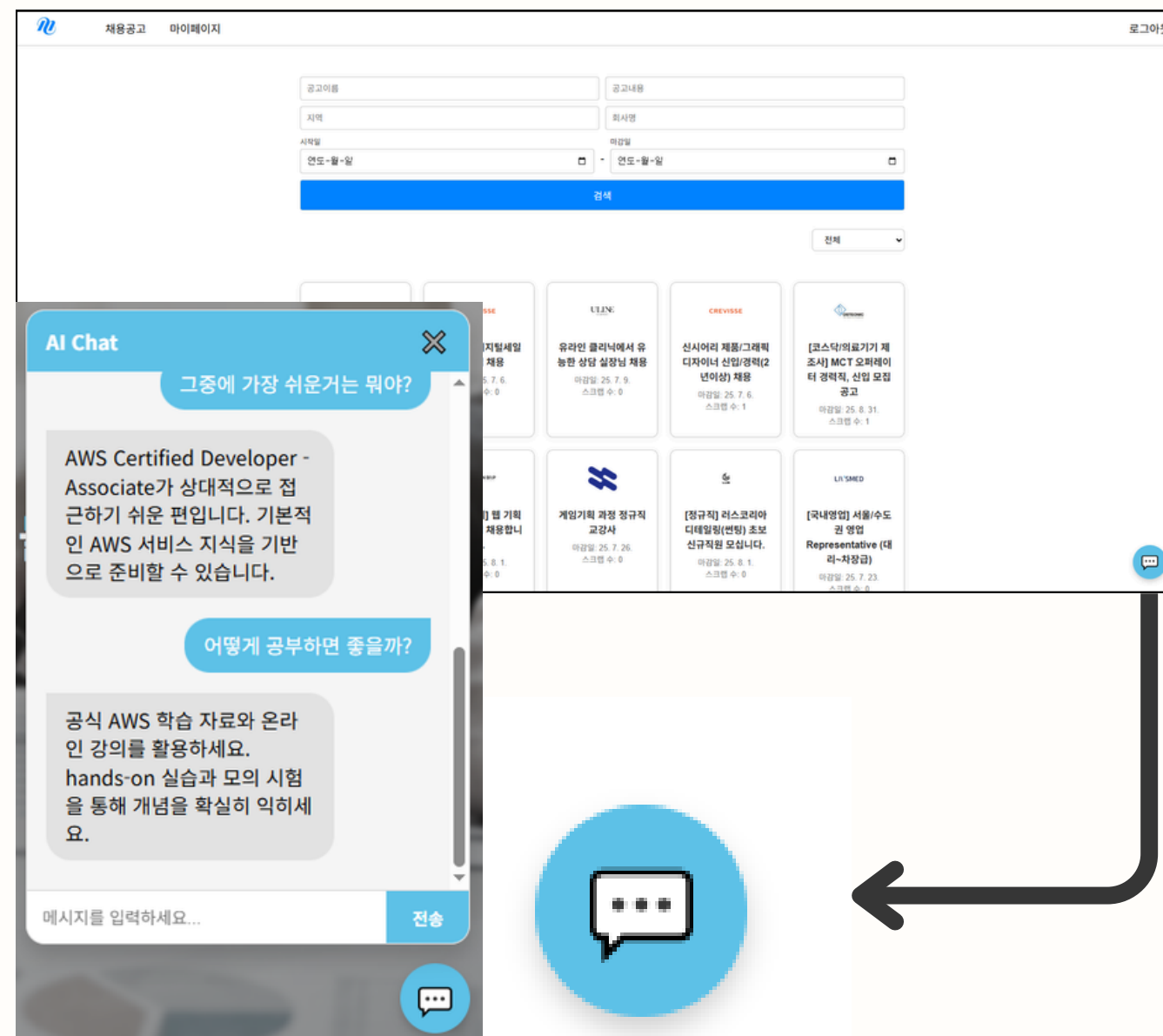
✓ 최근 7일 검색 키워드 TOP 5, 인기 직무 TOP 5 - 사용자 대시보드와 동일



4. 프로젝트 수행 경과 챗봇

사용자 메시지 입력 → OpenAI API 호출 → AI 응답 수신

✓ 챗봇



- 사용자의 질문 입력 처리 및 챗봇 답변 출력
- OpenAI 등 외부 AI API 연동 (REST API)
- 서버 오류 / 네트워크 오류 시 오류 메시지를 챗봇 메시지로 표시
- 최신 메시지가 항상 보이도록 스크롤 처리
- 엔터 키로 메시지 전송 가능 (keydown 이벤트)
- 채팅 토글 버튼 클릭 시 챗봇 팝업 열기/닫기 (toggleChat())
- 로그인 성공 시 챗봇 사용 가능

4. 프로젝트 수행 경과

챗봇

사용자 메시지 입력 → OpenAI API 호출 → AI 응답 수신

사용자 메시지 입력해서 서버에 전달하는 코드

main.html

```
<!-- 채팅 토글 버튼 -->
<div class="chat-toggle-button" onclick="toggleChat()"><img alt="chat icon" data-bbox="265 350 278 365"/></div>

<!-- 채팅 팝업 -->
<div class="chat-popup" id="chatPopup">
  <div class="chat-header">
    <span class="chat-title">AI Chat</span>
    <button class="close-btn" onclick="toggleChat()"><img alt="close icon" data-bbox="288 475 301 490"/></button>
  </div>

  <!-- 메시지 영역 -->
  <div class="chat-box" id="chatBox">
    <!-- 여기에 메시지들이 들어감 -->
  </div>

  <!-- 입력창은 형제로 따로! -->
  <div class="input-box">
    <input type="text" id="userInput" placeholder="메시지를 입력하세요." />
    <button onclick="sendMessage()">전송</button>
  </div>
</div>
```

```
<script>
function toggleChat() {
  const popup = document.getElementById("chatPopup");
  popup.style.display = (popup.style.display === "block") ? "none" : "block";
}

async function sendMessage() {
  const inputEl = document.getElementById("userInput");
  const message = inputEl.value.trim();
  if (!message) return;

  const chatBox = document.getElementById("chatBox");

  // 유저 메시지 추가
  const userMsg = document.createElement("div");
  userMsg.className = "message user-message";
  userMsg.textContent = message;
  chatBox.appendChild(userMsg);

  inputEl.value = "";

  // 작성중 메시지 추가
  const botMsg = document.createElement("div");
  botMsg.className = "message bot-message";
  botMsg.textContent = "... 작성중입니다";
  chatBox.appendChild(botMsg);
  chatBox.scrollTop = chatBox.scrollHeight;
}
```

- fetch()로 /api/chat POST 요청
→ body에 { input: message } JSON 담아 전송
→ headers에 CSRF 토큰 포함

```
try {
  const response = await fetch("/api/chat", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      [csrfHeader]: csrfToken, // CSRF 토큰 동적 헤더 적용
    },
    body: JSON.stringify({ input: message }),
  });

  if (response.status === 403) {
    alert("로그인이 필요한 서비스입니다.\n로그인 페이지로 이동합니다.");
    window.location.href = "/user/login";
    return;
  }

  if (!response.ok) {
    const errorText = await response.text();
    console.error("서버 오류:", errorText);
    botMsg.classList.add("error-message");
    botMsg.textContent = "서버 오류가 발생했어요.\n" + errorText;
    chatBox.scrollTop = chatBox.scrollHeight;
    return;
  }
}
```

- 사용자가 #userInput에 메시지를 입력하고 전송 버튼을 누름.
- 버튼 클릭 시 sendMessage() 함수가 실행됨.

- 사용자 입력 읽기
- 채팅창에 사용자 메시지 출력
- 로딩 메시지 출력

4. 프로젝트 수행 경과

챗봇

사용자 메시지 입력 → OpenAI API 호출 → AI 응답 수신

사용자 메시지 입력해서 서버에 전달하는 코드

ChatController

```
@RestController no usages 👤 tjoeunJava +1
@RequestMapping("/api")
@RequiredArgsConstructor
public class ChatController {

    private final ChatService chatService;

    @PostMapping("/chat") no usages 👤 tjoeunJava +1
    public ChatResponse handleChat(@RequestBody ChatRequest request, Principal principal) {
        String input = request.getInput();
        String userId = (principal != null) ? principal.getName() : "anonymous";
        String reply = chatService.generateReply(userId, input);
        return new ChatResponse(reply);
    }
}
```

- @RequestBody로 JSON 메시지 받음
- ChatService에 넘겨 GPT 호출
- GPT 호출하고 DB 저장 후 reply 반환

ChatService

```
public String generateReply(String userId, String userInput) { 1 usage 👤 tjoeunJava
    // 사용자 입력
    String prompt = userInput;

    // 과거 대화 불러오기
    List<ChatHistory> historyList = chatHistoryRepository
        .findTop10ByUserIdOrderByCreatedAtAsc(userId);

    List<Map<String, String>> messages = new ArrayList<>();
    messages.add(Map.of(
        k1: "role", v1: "system",
        k2: "content", v2: "You are a helpful assistant. " +
            "모든 답변은 반드시 한국어로 100자 이내로 핵심만 요약해서 제공해줘."
    ));

    for (ChatHistory h : historyList) {
        messages.add(Map.of( k1: "role", h.getRole(), k2: "content", h.getContent()));
    }

    messages.add(Map.of( k1: "role", v1: "user", k2: "content", prompt));
}
```

- 과거 대화 불러오기
- GPT 요청 payload 만들기

4. 프로젝트 수행 경과

챗봇

사용자 메시지 입력 → **OpenAI API 호출** → AI 응답 수신

왜 OpenAI API를 선택했는가?

- Hugging Face API 무료
 - => 대화형 모델 X (분류나 단일 요약 모델)
 - => 실제 챗봇처럼 대화 유지가 어려움
- Gemini API(Google) 유료
 - => 대화형은 가능/ GCP(구글 클라우드) 프로젝트를 설정
 - => 외부 클라우드로 연결해야 함
 - => 설정과 관리가 복잡
- OpenAI API 유료
 - => 공식 REST API로 바로 호출 가능 (별도 클라우드 배포 X)
 - => 텍스트 생성, 요약, 번역 등 다양한 작업 가능
 - => 대화 맥락 잘 이해해 대화할 수 있음 (대화형 챗봇 구현에 적합)



HUGGING FACE

Gemini



4. 프로젝트 수행 경과

챗봇

사용자 메시지 입력 → OpenAI API 호출 → **AI 응답 수신**

AI 응답이 어떻게 수신되는지 알려주는 코드

- RestTemplate으로 OpenAI API 호출

```
RestTemplate restTemplate = new RestTemplate();

HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(openaiApiKey);
headers.setContentType(MediaType.APPLICATION_JSON);
headers.setAccept(Collections.singletonList(MediaType.APPLICATION_JSON));

Map<String, Object> payload = Map.of(
    k1: "model", v1: "gpt-4.1-nano",
    k2: "messages", messages
);

HttpEntity<Map<String, Object>> request = new HttpEntity<>(payload, headers);
```

ChatService

```
private static final String OPENAI_API_URL = "https://api.openai.com/v1/chat/completions"; 1 usage

try {
    ResponseEntity<String> response = restTemplate.postForEntity(
        OPENAI_API_URL, request, String.class);

    MediaType contentType = response.getHeaders().getContentType();
    if (contentType == null || !contentType.includes(MediaType.APPLICATION_JSON)) {
        return "OpenAI 서버 오류 또는 유효하지 않은 응답 형식입니다.";
    }
}
```

- OpenAI에 POST 요청 보내고 응답 받기

4. 프로젝트 수행 경과

챗봇

사용자 메시지 입력 → OpenAI API 호출 → **AI 응답 수신**

AI 응답이 어떻게 수신되는지 알려주는 코드

ChatService

```
String reply = root.get("choices")
    .get(0)
    .get("message")
    .get("content")
    .asText();

// DB 저장
chatHistoryRepository.saveAll(List.of(
    ChatHistory.builder().userId(userId).role("user").content(prompt)
        .createdAt(LocalDateTime.now()).build(),
    ChatHistory.builder().userId(userId).role("assistant").content(reply)
        .createdAt(LocalDateTime.now()).build()
));

return reply.trim();
```

- 응답 파싱 RestTemplate으로 OpenAI 호출
- 사용자/AI 메시지 DB 저장
- 요약 답변 반환
- DB 저장

개발자의 꿀팁 알려줘!

코드 정리 습관, 버전 관리, 꾸준한 공부와 협업으로 개발 효율 높이기!

4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃 → 회원정보 열람/수정/삭제

✓ 회원가입

회원가입

이름

이메일

닉네임

비밀번호

비밀번호 확인

연도 - 월 - 일

중복확인

중복확인

중복확인

회원가입

회원가입

더조은

tjoeun@naver.com

중복확인

사용 가능한 이메일입니다.

더조은12

중복확인

사용 가능한 닉네임입니다.

.....

.....

2009-01-01

회원가입

localhost 내용:

이메일 중복 확인을 해주세요.

확인

.....

.....

연도-월-일

생년월일은 필수 입력 값입니다.

회원가입

- 이메일 / 닉네임 중복 확인 (AJAX + REST API)
- 서버 유효성 검사 (@Valid, @NotBlank, @Email, etc.)
- 비밀번호 암호화 저장 (PasswordEncoder)

4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃 →
회원정보 열람 / 수정 / 삭제

✓ 사용자가 회원가입 정보 입력

=> 클라이언트 측 유효성 검사(Javascript)

- 이메일 닉네임 중복 확인(/api/user/check-email, /check-nickname)
- 비밀번호 일치 여부 확인
- 비밀번호 유효성 검사

```
let emailChecked = false;
let nicknameChecked = false;

async function checkEmail() {
  const email = document.getElementById("userEmail").value.trim();
  const msg = document.getElementById("emailMessage");
  if (!email) {
    msg.textContent = "이메일을 입력해주세요.";
    emailChecked = false;
    return;
  }
  const res = await fetch(`/api/user/check-email?email=${email}`);
  const data = await res.json();
  if (data.exists) {
    msg.textContent = "이미 존재하는 이메일입니다.";
    emailChecked = false;
  } else {
    msg.textContent = "사용 가능한 이메일입니다.";
    emailChecked = true;
  }
}
```

```
function validateForm() {
  const pw = document.getElementById("userPassword").value;
  const pwConfirm = document.getElementById("confirmPassword").value;

  if (!emailChecked) {
    alert("이메일 중복 확인을 해주세요.");
    return false;
  }
  if (!nicknameChecked) {
    alert("닉네임 중복 확인을 해주세요.");
    return false;
  }
  if (pw !== pwConfirm) {
    alert("비밀번호가 일치하지 않습니다.");
    return false;
  }
  return true;
}
```

signup.html

```
@GetMapping("/check-email") no usages ② tjoeunJava
public ResponseEntity<?> checkEmail(@RequestParam String email) {
  boolean exists = userService.emailExists(email);
  return ResponseEntity.ok(Collections.singletonMap("exists", exists));
}
```

```
@GetMapping("/check-nickname") no usages ② tjoeunJava
public ResponseEntity<?> checkNickname(@RequestParam String nickname) {
  boolean exists = userService.nicknameExists(nickname);
  return ResponseEntity.ok(Collections.singletonMap("exists", exists));
}
```

UserRestController

4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃
→ 회원정보 열람 / 수정 / 삭제

=> 서버 측 유효성 검사

- DTO 검증 : 이메일, 닉네임, 비밀번호, 생년월일 등 각 필드에 유효성 검사 진행

```
@Getter @Setter 17 usages  tjoenJava
public class UserFormDto {

    @NotBlank(message = "이름은 필수 입력 값입니다.")
    @Size(min = 2, max = 10, message = "이름은 2~10글자 사이로 입력해주세요.")
    @Pattern(regexp = "^[가-힣a-zA-Z]+$", message = "이름은 특수문자와 숫자를 포함할 수 없습니다.")
    private String userName;

    @NotBlank(message = "이메일은 필수 입력 값입니다.")
    @Email(message = "이메일 형식으로 입력해주세요.")
    private String userEmail;

    @NotBlank(message = "닉네임은 필수 입력 값입니다.")
    @Size(min = 2, max = 20, message = "닉네임은 2~20글자 사이로 입력해주세요.")
    private String userNickname;

    @NotBlank(message = "비밀번호는 필수 입력 값입니다.")
    @Size(min = 8, max = 16, message = "비밀번호는 8~16글자 사이로 입력해주세요.")
    @Pattern(regexp = "^[a-zA-Z0-9]*$", message = "비밀번호는 한글과 특수문자를 포함할 수 없습니다.")
    private String userPassword;

    @NotNull(message = "생년월일은 필수 입력 값입니다.")
    private LocalDate userBirth;

    private UserRole userRole;
}
```

✓ Role 권한 부여

- User 엔티티 객체 생성, 비밀번호는 암호화

```
public static Users createUser(UserFormDto dto, PasswordEncoder passwordEncoder) {
    return Users.builder()
        .userName(dto.getUserName())
        .userEmail(dto.getUserEmail())
        .userPassword(passwordEncoder.encode(dto.getUserPassword()))
        .userNickname(dto.getUserNickname())
        .userBirth(dto.getUserBirth())
        .userCreatedAt(new Timestamp(System.currentTimeMillis()))
        .userRole(UserRole.USER)
        .build();
}
```

- UserRole enum을 통해 USER, ADMIN 권한을 정의
- Spring Security의 접근 제어에 활용

```
public enum UserRole {
    USER, ADMIN 1 usage
}
```


4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃
→ 회원정보 열람 / 수정 / 삭제

✓ DB + Elasticsearch에 색인

=> 서버 측 이메일/닉네임 중복 검사 후 User 생성 및 DB 저장
UserService

```
public Users saveUser(Users user) { 1 usage  ⚙ green4141 +1 *
    validateDuplicateUser(user);
    Users savedUser = userRepository.save(user);
    UserDocument userDoc = toUserDocument(savedUser);
    userDocumentRepository.save(userDoc);
    return savedUser;
}

private UserDocument toUserDocument(Users user) { 1 usage  ⚙ green4141
    return UserDocument.builder()
        .userIdx(user.getUserIdx())
        .userName(user.getUserName())
        .userEmail(user.getUserEmail())
        .userNickname(user.getUserNickname())
        .userBirth(user.getUserBirth().atStartOfDay(ZoneId.systemDefault()).toInstant().toEpochMilli())
        .userRole(user.getUserRole().toString())
        .userCreatedAt(user.getUserCreatedAt())
        .build();
}

private void validateDuplicateUser(Users user) { 1 usage  ⚙ tjoeunJava
    if (userRepository.findByUserEmail(user.getUserEmail()) != null) {
        throw new IllegalStateException("이미 가입된 이메일입니다.");
    }
    if (userRepository.findByUserNickname(user.getUserNickname()) != null) {
        throw new IllegalStateException("이미 사용 중인 닉네임입니다.");
    }
}
```

=> 사용자 데이터를 Elasticsearch에 색인

UserRepository

```
public interface UserRepository extends JpaRepository<Users, Integer> {
    Users findByUserEmail(String email); 19 usages  ⚙ tjoeunJava
    Users findByUserNickname(String nickname); 3 usages  ⚙ tjoeunJava
}
```

✓ DB 저장 및 Elasticsearch 연동 확인

GET users/_doc/3

```
{
  "_index": "users",
  "_id": "3",
  "_version": 2,
  "_seq_no": 2,
  "_primary_term": 1,
  "found": true,
  "_source": {
    "_class": "com.tjoeun.elasticsearch.document.UserDocument",
    "userIdx": 3,
    "userName": "사용자",
    "userEmail": "user@naver.com",
    "userNickname": "사용자",
    "userBirth": 1751554800000,
    "userRole": "USER",
    "userCreatedAt": "2025-07-15T12:41:41.000Z"
  }
}
```

user_idx	user_birth	user_created_at	user_email	user_name	user_nickname	user_password	user_role
3	2025-07-04	2025-07-15 21:41:41	user@naver.com	사용자	사용자	\$2a\$10\$Xi0FSHDiYg.BixLNZ1l1yexvflBUjDXEqyGrI7fDDaJgyzQIw2qxm	USER

4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃
→ 회원정보 열람 / 수정 / 삭제

✓ 로그인/로그아웃

로그인

이메일

비밀번호

로그인

회원가입

로그인

이메일

비밀번호

이메일 또는 비밀번호를 확인해주세요

로그인

회원가입

- 폼 기반 로그인 처리 (/user/login)
- 비밀번호 암호화 비교
- 로그인 실패 시 사용자 메시지 출력
- CSRF 토큰 처리
- 정상 로그인 시 메인 페이지("/")로 리다이렉트

4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃
→ 회원정보 열람 / 수정 / 삭제

login.html

- POST 방식으로 /user/login 요청 전송 : 이 요청은 Spring Security로 넘어감

```
<form th:action="@{/user/login}" method="post" class="signup-form" novalidate>
  <input type="email" name="userEmail" placeholder="이메일" autocomplete="username" />
  <input type="password" name="password" placeholder="비밀번호" autocomplete="current-password" />
```

SecurityConfig

- userEmail을 필드를 기준으로 인증 진행 성공 시 /화면으로 리다이렉트

```
.authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
    .requestMatchers( ...patterns: "/css/**", "/js/**", "/images/**", "/static/**").permitAll()
    .requestMatchers( ...patterns: "/", "/user/signup", "/user/login", "/api/user/**", "/user/login/error").permitAll()
    .requestMatchers( ...patterns: "/empl/empl_main", "/empl/empl_detail/**").permitAll()
    .requestMatchers( ...patterns: "/admin/**").hasRole("ADMIN")
    .requestMatchers( ...patterns: "/mypage/**").authenticated() // 마이페이지는 로그인 사용자만
    .anyRequest().authenticated()
)
.formLogin( FormLoginConfigurer<HttpSecurity> form -> form
    .loginPage("/user/login")
    .defaultSuccessUrl( defaultSuccessUrl: "/", alwaysUse: true)
    .usernameParameter("userEmail")
    .failureHandler(new FormLoginAuthenticationFailureHandler())
)
.logout( LogoutConfigurer<HttpSecurity> logout -> logout
    .logoutRequestMatcher(new AntPathRequestMatcher( pattern: "/user/logout"))
    .logoutSuccessUrl("/")
)
.build();
```

- /user/singup, /user/login, /empl/empl_main 같은 경로는 누구나 접근 가능
- /mypage/** 경로는 로그인한 사용자만 접근 가능
- /admin/** 경로는 관리자만 접근 가능
- 그 외 모든 경로는 .anyRequest().authenticated()로 로그인 필요

```
<span sec:authorize="isAuthenticated()">
  <a th:href="@{/user/logout}">로그아웃</a>
</span>
```

4. 프로젝트 수행 경과

사용자 부분

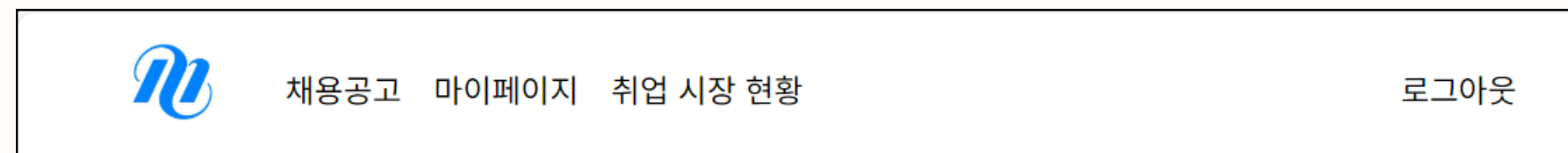
회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃
→ 회원정보 열람 / 수정 / 삭제

- Thymelef + Spring Security 로 권한에 따라 메뉴 보여주는 방식

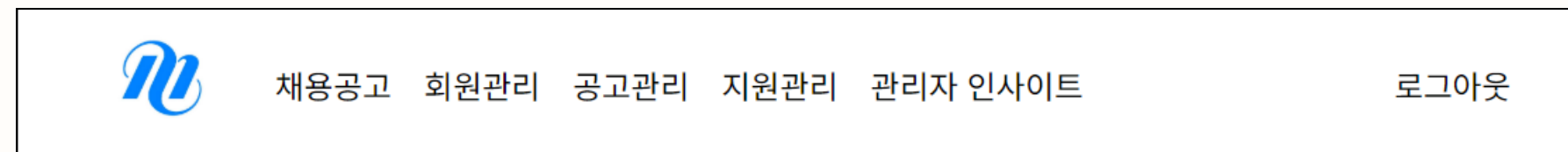
비회원



사용자



관리자



header.html

```
<span><a th:href="@{/empl/empl_main}">채용공고</a></span>

<span sec:authorize="hasAnyAuthority('ROLE_USER')">
  <a th:href="@{/mypage/member_modify}">마이페이지</a>
</span>

<span sec:authorize="hasAnyAuthority('ROLE_USER')">
  <a th:href="@{/dashboard/user}">취업 시장 현황</a>
</span>

<span sec:authorize="hasAnyAuthority('ROLE_ADMIN')">
  <a th:href="@{/admin/member_list}">회원관리</a>
</span>
<span sec:authorize="hasAnyAuthority('ROLE_ADMIN')">
  <a th:href="@{/admin/recruit_list}">공고관리</a>
</span>
<span sec:authorize="hasAnyAuthority('ROLE_ADMIN')">
  <a th:href="@{/admin/apply_list}">지원관리</a>
</span>
<span sec:authorize="hasAnyAuthority('ROLE_ADMIN')">
  <a th:href="@{/dashboard/admin/dashboard}">관리자 인사이드</a>
</span>
</div>
<div class="menu right">

  <span sec:authorize="isAnonymous()">
    <a th:href="@{/user/login}">로그인</a>
  </span>
  <span sec:authorize="isAnonymous()">
    <a th:href="@{/user/signup}">회원가입</a>
  </span>
  <span sec:authorize="isAuthenticated()">
    <a th:href="@{/user/logout}">로그아웃</a>
  </span>
</div>
```


4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃
→ **회원정보 열람 / 수정 / 삭제**

✓ 회원 정보

마이페이지

회원정보수정

이력서관리

지원현황

스크랩

이름

사용자

이메일

user@naver.com

닉네임

사용자2222

중복확인

비밀번호

비밀번호 입력 (변경 시에만)

비밀번호 확인

비밀번호 확인

생년월일

2025-07-09

수정

회원 탈퇴

- 회원정보 수정 (닉네임 + 비밀번호) 각각 or 함께
- 유효성 검증
- 회원 탈퇴

닉네임

사용자

중복확인

사용 가능한 닉네임입니다.

localhost 내용:

회원정보가 성공적으로 수정되었습니다.

확인

닉네임

사용자

중복확인

localhost 내용:

정말 탈퇴하시겠습니까?

확인

취소

승리는 가장 끈기 있는 자에게 돌아간다.

4. 프로젝트 수행 경과

사용자 부분

회원가입 → DB/Elasticsearch 색인 → 로그인/로그아웃
→ 회원정보 열람 / 수정 / 삭제

✓ 회원 정보 열람

```
@Transactional(readOnly = true) 1 usage
public UserFormDto getUserFormDto(String email) {
    Users user = userRepository.findByUserEmail(email);
    if (user == null) {
        throw new IllegalArgumentException("로그인한 사용자를 찾을 수 없습니다.");
    }
    UserFormDto dto = new UserFormDto();
    dto.setUserName(user.getUserName());
    dto.setUserEmail(user.getUserEmail());
    dto.setUserNickname(user.getUserNickname());
    dto.setUserBirth(user.getUserBirth());
    dto.setUserPassword("");
    return dto;
}
```

UserSerivce

✓ 회원 정보 삭제

```
@Transactional 1 usage 2 tjoeunJava +1
public void deleteUserByEmail(String email) {
    Users user = userRepository.findByUserEmail(email);
    if (user != null) {
        userRepository.delete(user);
        userSyncService.deleteById(user.getUserIdx().longValue());
        applyHistorySyncService.deleteByUserId(user.getUserIdx());
    } else {
        throw new IllegalArgumentException("해당 이메일의 사용자가 존재하지 않습니다.");
    }
}
```

UserSerivce

✓ 회원 정보 수정

```
@Transactional 1 usage 2 tjoeunJava +1 *
public String updateUser(UserFormDto dto, String email, BindingResult bindingResult, Model model,
    PasswordEncoder passwordEncoder, UserService userService) {
    Users user = userRepository.findByUserEmail(email);
    if (user == null) {
        throw new IllegalArgumentException("사용자를 찾을 수 없습니다.");
    }
    if (!dto.getUserNickname().equals(user.getUserNickname()) &&
        userService.nicknameExists(dto.getUserNickname())) {
        model.addAttribute(attributeName: "nicknameError", attributeValue: "이미 사용 중인 닉네임입니다.");
        return "mypage/member_modify";
    }
    if (dto.getUserPassword() != null && !dto.getUserPassword().isBlank()) {
        user.setUserPassword(passwordEncoder.encode(dto.getUserPassword()));
    }
    user.setUserNickname(dto.getUserNickname());
    userRepository.save(user);
    userSyncService.save(user);
    applyHistorySyncService.updateUserInfoInES(user);
    return "redirect:/mypage/member_modify?success";
}
```

UserSerivce

- 현재 로그인한 사용자의 정보를 가져와 수정 화면에 표시할 DTO로 변환
- 사용자 정보 업데이트 후 DB 저장, Elasticsearch의 유저 정보 (userSyncService, applyHistorySyncService)도 동기화
- 해당 이메일의 사용자를 DB와 Elasticsearch 양쪽에서 완전히 삭제

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → 이력서 작성 → 이력서 읽기/삭제 → 이력서 수정

✓ 이력서 관리

마이페이지

회원정보수정

이력서관리

지원현황

스크랩

추가

자소서 예비용2

삭제

자소서 예비용

삭제

피아니스트

삭제

웹 디자이너

삭제

백엔드 개발자

삭제

1

2

>

- 이력서 목록 조회
- 이력서 상세 이동(이력서 제목 클릭 시 상세 페이지(/react_detail/{id})로 이동)
- 이력서 추가 버튼(새 이력서를 작성할 수 있는 페이지(/react_write)로 이동)
- 페이지네이션 처리

localhost 내용:
정말 삭제하시겠습니까?

확인

취소

마이페이지

회원정보수정

이력서관리

지원현황

스크랩

추가

자소서 예비용2

삭제

마이페이지

회원정보수정

이력서관리

지원현황

스크랩

추가

자소서 예비용

삭제

피아니스트

삭제

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → 이력서 작성 → 이력서 읽기/삭제 → 이력서 수정

✓ 이력서 리스트

MyPageController

```
@GetMapping("/react_list") no usages
public String reactList(@RequestParam(defaultValue = "1") int page,
                        @PageableDefault(size = 5) Pageable pageable,
                        Model model,
                        Principal principal) {
    String email = principal.getName();
    Users user = userRepository.findByUserEmail(email);
    Pageable correctedPageable = PageRequest.of(
        pageNumber: page - 1,
        pageable.getPageSize(),
        Sort.by(Sort.Direction.DESC, ...properties: "updatedAt")
    );
    Page<Resume> resumePage = myPageService.getPagedResumesByUserId(user.getUserIdx().longValue(),
        correctedPageable);
    model.addAttribute(attributeName: "resumes", resumePage.getContent());
    model.addAttribute(attributeName: "jobPage", resumePage);
    PaginationUtil.setPaging(model, resumePage, url: "/mypage/react_list");
    return "mypage/react_list";
}
```

MyPageService

```
public Page<Resume> getPagedResumesByUserId(Long userIdx, Pageable pageable) {
    return resumeRepository.findByUser_UserIdx(userIdx, pageable);
}
```

✓ 이력서 삭제

MyPageController

```
@PostMapping("/react_delete/{id}") no usages
@Transactional
public String deleteResume(@PathVariable Long id,
                           @RequestParam(defaultValue = "1") int page) {
    resumeRepository.deleteById(id);

    long totalCount = resumeRepository.count();
    int pageSize = 5;
    int lastPage = (int) Math.ceil((double) totalCount / pageSize);

    if (lastPage < 1) {lastPage = 1;}
    if (page > lastPage) {page = lastPage;}
    if (page < 1) {page = 1;}
    return "redirect:/mypage/react_list?page=" + page;
}
```

- reactList() 에서 로그인한 사용자의 이력서를 Page<Resume> 로 가져옴.
- model.addAttribute("resumes", ...) 로 뷰에 이력서 리스트 전달.
- Thymeleaf 에서 th:each="resume : \${resumes}" 로 반복 출력.

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → **이력서 작성** → 이력서 읽기/삭제 → 이력서 수정

✓ 이력서 작성

마이페이지 > 이력서

* 제목	제목을 입력해 주세요.	
이름	사용자	
이메일	user@naver.com	
생년월일	2025-07-09	
* 주소	주소를 입력해 주세요.	
* 전화번호	전화번호를 입력해 주세요. ex. 010-0000-0000	
학력	학력을 입력해 주세요.	
자격증 및 스킬	자격증 및 스킬을 입력해 주세요.	
경력	경력을 입력해 주세요.	
수상이력	수상이력을 입력해 주세요.	

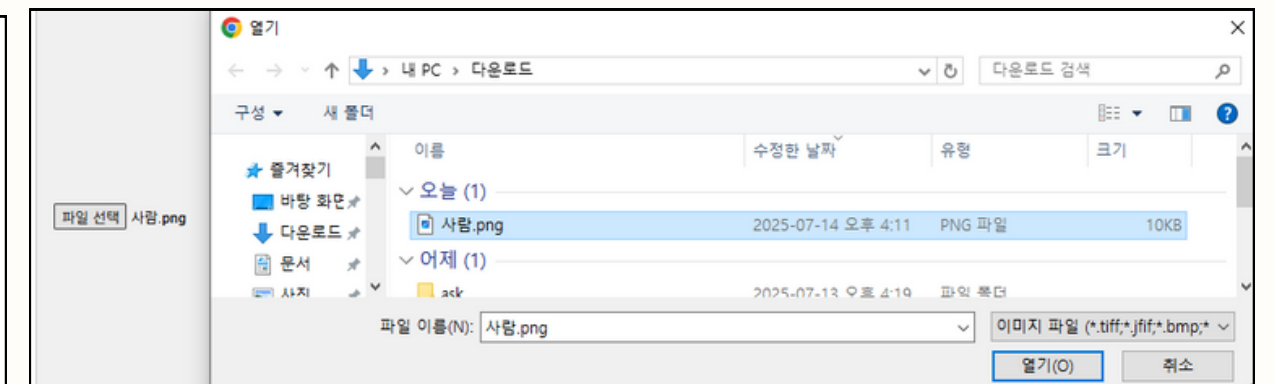
파일 선택

선택된 파일 없음

* 질문 1을 입력하세요.	* 답변을 입력하세요.
질문 2을 입력하세요.	답변을 입력하세요.
질문 3을 입력하세요.	답변을 입력하세요.
질문 4을 입력하세요.	답변을 입력하세요.
포트폴리오 링크를 넣어주세요.	예: https://github.com/yourname/portfolio

취소 저장

- 이력서 등록 폼 작성
- 사용자 기존 정보 불러옴
- 필수 입력 항목 유효성 검사 (JavaScript)
- 질문/답변 최대 4쌍까지 작성 가능
- 이미지 업로드 미리보기
- 저장 및 취소 버튼 처리



localhost 내용:
제목을 작성해 주세요!

확인

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → **이력서 작성** → 이력서 읽기/삭제 → 이력서 수정

✓ 이력서 작성

MyPageController

```
@GetMapping("/react_write") no usages 2 tjoeunJava +1
public String reactWrite(Model model, Principal principal) {
    Users user = userRepository.findByUserEmail(principal.getName());
    model.addAttribute(attributeName: "user", user);
    model.addAttribute(attributeName: "formDto", new ResumeDto());
    return "mypage/react_write";
}
```

- 로그인한 사용자의 이름, 이메일, 생년월일을 읽기 전용으로 보여주기 위해 user 모델을 뷰에 전달
- 새 이력서 작성 폼은 빈 ResumeDto 로 초기화됨

```
@PostMapping("/react_write") no usages 2 tjoeunJava +5 *
public String saveResume(
    @ModelAttribute("resumeDto") @Valid ResumeDto resumeDto,
    @RequestParam("imgFile") MultipartFile imgFile,
    BindingResult bindingResult,
    Principal principal) throws IOException {

    if (resumeDto.getTitle() != null && resumeDto.getTitle().length() > 45) {
        bindingResult.rejectValue(field: "title", errorCode: "length",
            defaultMessage: "제목은 최대 45자까지 입력할 수 있습니다.");
    }

    if (bindingResult.hasErrors()) {
        return "mypage/react_write";
    }

    String email = principal.getName();
    Users user = userRepository.findByUserEmail(email);

    if (user == null) {
        throw new IllegalArgumentException("로그인한 사용자를 찾을 수 없습니다.");
    }

    Timestamp now = new Timestamp(System.currentTimeMillis());
    Resume resume = new Resume();
    resume.setUser(user);
    resume.setTitle(resumeDto.getTitle());
    resume.setAddress(resumeDto.getAddress());
    resume.setPhoneNum(resumeDto.getPhoneNum());
    resume.setEducation(resumeDto.getEducation());
    resume.setAbility(resumeDto.getAbility());
    resume.setAntecedents(resumeDto.getAntecedents());
    resume.setAwards(resumeDto.getAwards());
    resume.setContext(resumeDto.getContext());
    resume.setCreatedAt(now);
    resume.setUpdatedAt(now);
}
```

```
function validateForm() {
    const title = document.getElementById('title').value.trim();
    const address = document.getElementById('address').value.trim();
    const phoneNum = document.getElementById('phone_num').value.trim();
    const question1 = document.querySelector('textarea[name="resumeContents[0].question"]').value.trim();
    const context1 = document.querySelector('textarea[name="resumeContents[0].context"]').value.trim();

    if (!title) { alert('제목을 작성해 주세요!'); return false; }
    if (!address) { alert('주소를 작성해 주세요!'); return false; }
    if (!phoneNum) { alert('전화번호를 작성해 주세요!'); return false; }
    if (!question1) { alert('질문 1을 작성해 주세요!'); return false; }
    if (!context1) { alert('답변 1을 작성해 주세요!'); return false; }

    for (let i = 0; i < 4; i++) {
        const questionEl = document.querySelector(`textarea[name="resumeContents[${i}].question"]`);
        const contextEl = document.querySelector(`textarea[name="resumeContents[${i}].context"]`);

        const question = questionEl.value.trim();
        const context = contextEl.value.trim();

        if (!question && context) {
            alert(`질문 ${i + 1}이 비어있습니다. 질문을 작성해 주세요!`);
            questionEl.focus();
            return false;
        }
        if (question && !context) {
            alert(`질문 ${i + 1}에 대한 답변을 작성해 주세요!`);
            contextEl.focus();
            return false;
        }
    }

    return true; // 통과 시 제출 허용
}
```

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → 이력서 작성 → **이력서 읽기/삭제** → 이력서 수정

✓ 이력서 읽기/삭제

마이페이지 > 이력서

제목	폴스텍 개발자
 이름 사용자 이메일 user@naver.com 생년월일 2025-07-09 주소 서울특별시 00구 000로 00길 00 000동 000호 전화번호 010-1234-1234 학력 00대학 졸업 자격증 및 스킬 00000기사 자격증 경력 0000회사 3개월 인턴 경험	
본인을 한 단어로 표현한다면, 그리고 그 이유는 무엇인가요?	저를 한 단어로 표현한다면 '성실함'입니다. 저는 주어진 일에 책임감을 갖고 끝까지 완수하려는 태도를 항상 중요하게 여깁니다. 대학교 팀 프로젝트에서는 맡은 역할 외에도 전체 일정 관리와 자료 정리를 도맡으며 팀의 완성도를 높였습니다. 덕분에 팀은 학기 최우수 프로젝트로 선정될 수 있었고, 이 경험을 통해 성실함이 단순한 노력 이상의 가치를 가진다는 것을 배웠습니다. 조직 내에서 신뢰받는 사람이 되는 것이 저의 목표이며, 성실함은 이를 이루는 핵심 역량이라 믿습니다.
가장 어려웠던 경험과 그것을 극복한 방법은 무엇인가요?	대학교 3학년 때 전공 수업에서 난이도가 높은 데이터 분석 프로젝트를 맡았을 때가 가장 어려웠던 경험이었습니다. 팀원 모두 Python 사용 경험이 부족했고, 처음에는 분석 방향도 잡기 어려웠습니다. 저는 팀을 대표해 직접 교수님과 상담하고, 온라인 강의를 찾아 공부하면서 기초부터 다져갔습니다. 이후 팀원들과 자료를 공유하고 간단한 코드부터 함께 실습하며 협업을 이어갔습니다. 결과적으로 프로젝트를 성공적으로 마무리했고, 발표에서도 높은 평가를 받았습니다. 이 경험을 통해 어려움은 회피보다 '구체적인 행동'으로 풀어가야 한다는 교훈을 얻었습니다.
지원한 직무에 본인이 적합한 이유는 무엇인가요?	저는 데이터 기반의 분석과 문제 해결에 강점을 가지고 있어, 귀사에서 요구하는 기획/분석 직무에 적합하다고 생각합니다. 대학 시절 마케팅 조사 동아리에서 실제 브랜드의 소비자 분석 프로젝트를 진행하며, 데이터를 기반으로 한 인사이트 도출 경험을 쌓았습니다. 특히 고객 설문 설계부터 SPSS 분석, 결과 보고서 작성까지 직접 주도하며 실무 감각을 익혔습니다. 또한, 인턴십에서는 고객 불만 데이터를 분석하여 서비스 개선안을 제안해 팀의 업무 효율을 높인 경험도 있습니다. 데이터를 이해하고 행동으로 옮기는 능력은 이 직무에 있어 중요한 역량이라 확신합니다.

목록보기

삭제

수정

- 이력서 전체 정보 출력 (제목, 이미지, 개인정보, 이력 정보 등)
- 입력한 자기소개 질문/답변 리스트 렌더링
- 질문/답변 최대 4쌍까지 작성 가능
- 작성하지 않은 빈 값은 화면에 출력하지 않고 조건부로 숨김 처리
- 이전 페이지로 이동: 이전 URL이 있을 경우 해당 경로로, 없으면 기본적으로 react_list로 복귀
- 수정/삭제/목록보기 버튼

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → 이력서 작성 → **이력서 읽기/삭제** → 이력서 수정

✓ 이력서 읽기/삭제

MyPageController

```
@GetMapping("/react_detail/{id}") no usages
public String reactDetail(@PathVariable Long id,
                          @RequestParam(defaultValue = "1") int page,
                          @RequestParam(defaultValue = "false") boolean readonly,
                          @RequestParam(required = false) String prevUrl,
                          Model model) {
    Resume resume = resumeRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("해당 이력서를 찾을 수 없습니다."));

    model.addAttribute(attributeName: "resume", resume);
    model.addAttribute(attributeName: "page", page);
    model.addAttribute(attributeName: "readonly", readonly);
    model.addAttribute(attributeName: "prevUrl", prevUrl);
    return "mypage/react_detail";
}
```

- id로 DB에서 Resume 단건 조회 → 못 찾으면 예외
- 모델에 resume / page / readonly / prevUrl을 담아 뷰에 전달

- DB에서 이력서 삭제
- 남은 총 건수 기준으로 페이지가 존재하지 않으면 마지막 페이지로 보정
→ 리스트가 깨지지 않음
- 삭제 후 다시 react_list로 이동

```
@PostMapping("/react_delete/{id}") no usages 2 tjoeunJava +1
@Transactional
public String deleteResume(@PathVariable Long id,
                           @RequestParam(defaultValue = "1") int page) {
    resumeRepository.deleteById(id);

    long totalCount = resumeRepository.count();
    int pageSize = 5;
    int lastPage = (int) Math.ceil((double) totalCount / pageSize);

    if (lastPage < 1) {
        lastPage = 1;
    }
    if (page > lastPage) {
        page = lastPage;
    }
    if (page < 1) {
        page = 1;
    }
    return "redirect:/mypage/react_list?page=" + page;
}
```

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → 이력서 작성 → 이력서 읽기/삭제 → **이력서 수정**

✓ 이력서 수정

	경력 0000회사 3개월 인턴 경험 ###아르바이트 2개월 수상이력 졸업작품 최우수
본인을 한 단어로 표현한다면, 그리고 그 이유는 무엇인가요?	저를 한 단어로 표현한다면 '성실함'입니다. 저는 주어진 일에 책임감을 갖고 끝까지 완수하려는 태도를 항상 중요하게 여깁니다. 대학교 팀 프로젝트에서는 맡은 역할 외에도 전체 일정 관리와 자료 정리를 도맡으며 팀의 완성도를 높였습니다. 덕분에 팀은 학기 최우수 프로젝트로 선정될 수 있었고, 이 경험을 통해 성실함이 단순한 노력 이상의 가치를 가진다는 것을 배웠습니다. 조직 내에서 신뢰받는 사람이 되는 것이 저의 목표이며, 성실함은 이를 이루는 핵심 역량이라 믿습니다.
가장 어려웠던 경험과 그것을 극복한 방법은 무엇인가요?	대학교 3학년 때 전공 수업에서 난이도가 높은 데이터 분석 프로젝트를 맡았을 때가 가장 어려웠던 경험이었습니다. 팀원 모두 Python 사용 경험이 부족했고, 처음에는 분석 방향도 잡기 어려웠습니다. 저는 팀을 대표해 직접 교수님과 상담하고, 온라인 강의를 찾아 공부하면서 기초부터 다져갔습니다. 이후 팀원들과 자료를 공유하고 간단한 코드부터 함께 실습하며 협업을 이어갔습니다. 결과적으로 프로젝트를 성공적으로 마무리했고, 발표에서도 높은 평가를 받았습니다. 이 경험을 통해 어려움은 회피보다 '구체적인 행동'으로 풀어가야 한다는 교훈을 얻었습니다.
질문과 답변을 변경	기존에 있던 질문과 답변을 변경해보겠습니다.
추가적인 질문 사항	새로운 질문과 답변을 추가해보았습니다.
포트폴리오 링크	https://github.com/RhoAias12/Project_tjeoun.git
취소 수정 완료	



제목	본스택 개발자
이름 사용자 이메일 user@naver.com 생년월일 2025-07-09 주소 서울특별시 00구 000로 00길 00 000동 000호 전화번호 010-1234-1234 학력 00대학 졸업 자격증 및 스킬 00000기사 자격증	경력 0000회사 3개월 인턴 경험 ###아르바이트 2개월 수상이력 졸업작품 최우수
본인을 한 단어로 표현한다면, 그리고 그 이유는 무엇인가요?	저를 한 단어로 표현한다면 '성실함'입니다. 저는 주어진 일에 책임감을 갖고 끝까지 완수하려는 태도를 항상 중요하게 여깁니다. 대학교 팀 프로젝트에서는 맡은 역할 외에도 전체 일정 관리와 자료 정리를 도맡으며 팀의 완성도를 높였습니다. 덕분에 팀은 학기 최우수 프로젝트로 선정될 수 있었고, 이 경험을 통해 성실함이 단순한 노력 이상의 가치를 가진다는 것을 배웠습니다. 조직 내에서 신뢰받는 사람이 되는 것이 저의 목표이며, 성실함은 이를 이루는 핵심 역량이라 믿습니다.
가장 어려웠던 경험과 그것을 극복한 방법은 무엇인가요?	대학교 3학년 때 전공 수업에서 난이도가 높은 데이터 분석 프로젝트를 맡았을 때가 가장 어려웠던 경험이었습니다. 팀원 모두 Python 사용 경험이 부족했고, 처음에는 분석 방향도 잡기 어려웠습니다. 저는 팀을 대표해 직접 교수님과 상담하고, 온라인 강의를 찾아 공부하면서 기초부터 다져갔습니다. 이후 팀원들과 자료를 공유하고 간단한 코드부터 함께 실습하며 협업을 이어갔습니다. 결과적으로 프로젝트를 성공적으로 마무리했고, 발표에서도 높은 평가를 받았습니다. 이 경험을 통해 어려움은 회피보다 '구체적인 행동'으로 풀어가야 한다는 교훈을 얻었습니다.
질문과 답변을 변경	기존에 있던 질문과 답변을 변경해보겠습니다.
추가적인 질문 사항	새로운 질문과 답변을 추가해보았습니다.
포트폴리오 링크	https://github.com/RhoAias12/Project_tjeoun.git
목록보기 삭제 수정	

- 기존 이력서 데이터를 불러와 수정할 수 있는 폼 제공
- 이미지 변경 기능
- 질문-답변 형식의 자기소개 항목 및 이외 항목 수정
- 필수 입력값에 대한 유효성 검사 (JavaScript 기반)
- 수정 취소 시 상세페이지로 복귀

4. 프로젝트 수행 경과

사용자 부분

이력서 관리 → 이력서 작성 → 이력서 읽기/삭제 → **이력서 수정**

✓ 이력서 수정

```
@GetMapping("/react_modify/{id}") no usages
public String reactModify(@PathVariable Long id,
                          @RequestParam(defaultValue = "1") int page,
                          Model model) {
    Resume resume = resumeRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("해당 이력서를 찾을 수 없습니다."));
    model.addAttribute(attributeName: "resume", resume);
    model.addAttribute(attributeName: "page", page);
    return "mypage/react_modify";
}

@PostMapping("/react_modify/{id}") no usages
public String updateResume(@PathVariable Long id,
                           @ModelAttribute ResumeDto resumeDto,
                           @RequestParam("imgFile") MultipartFile imgFile) throws IOException {
    myPageService.updateResume(id, resumeDto, imgFile);

    return "redirect:/mypage/react_detail/" + id;
}
```

MyPageController

- 수정할 Resume 데이터를 DB에서 조회
 - resume과 현재 페이지 page를 모델로 넘겨서 뷰에 렌더링
 - 뷰는 resume 값으로 기존 데이터를 채워서 사용자에게 보여줌
-
- id로 대상 Resume 가져와서 기존 데이터 수정
 - 새 이미지가 있으면 서버에 업로드 & 경로 저장
 - ResumeContent도 새로 바꿔서 저장
 - 저장 완료되면 해당 이력서 상세 페이지로 리다이렉트

4. 프로젝트 수행 경과

사용자 부분

채용공고 리스트 → 채용공고 상세(스크랩 / 지원하기) → 이력서 제출

✓ 채용공고

공고이름

공고내용

지역

회사명

시작일

마감일

연도-월-일

연도-월-일

검색

전체

전체

마감일: 최신순

마감일: 오래된순

스크랩: 많은순

스크랩: 적은순

자금 담당자 (Treasury manager)

상시채용

스크랩 수: 0

공공기술영업

상시채용

스크랩 수: 0

일만백만

회계/재무/총무 (서울)

상시채용

스크랩 수: 0

스마트 환경규제 및 폐수 관리 담당자

상시채용

스크랩 수: 0

웹 서버 개발자

상시채용

스크랩 수: 0

DevOps Engineer (1년 ~5년)

상시채용

스크랩 수: 0

DevOps Engineer (5년 이상)

상시채용

스크랩 수: 0

DBA 담당자

상시채용

스크랩 수: 0

[케어카운팅] 솔루션 B2B 영업 담당

상시채용

스크랩 수: 0

[이지케어] 재가요양 기관 ERP 영업 담당

마감일: 25. 7. 12.

스크랩 수: 1

- 다양한 검색 필터 제공(공고명, 공고내용, 지역, 회사명, 시작일, 마감일)
- 정렬 기능(마감일 기준, 스크랩 수 기준)
- 페이징 처리(PageRequest를 통한 페이지네이션 구현, 페이징 시 기존 검색/정렬 조건 유지)
- 공고 리스트 클릭 시 해당 상세 페이지로 이동
- 페이지네이션
- 검색 조건이 유지된 상태로 상세 페이지 → 목록으로 복귀 가능 (prevUrl 활용)
- 엘라스틱 검색 구현 참고

4. 프로젝트 수행 경과

사용자 부분

채용공고 리스트 → 채용공고 상세(스크랩 / 지원하기) → 이력서 제출

✓ 채용공고 상세

광고이름

지역

시작일

연도-월-일

검색

광고내용

회사명

마감일

연도-월-일

공공기술영업

네비웍스

상시채용 | 지역: 경기 안양시

Naviworks

고용형태

면접 후 결정

요구사항

자격요건

- 공공/국방 SI, NI 사업분야 경력 5~10년(과/차장급)
- ICT 사업기획/영업/제안/개발 업무 경험이 있으신 분
- 공공/국방 사업 제안서 작성 경력이 있으신 분
- 정보통신 분야 자격증을 보유하신 분

주요업무

- 주요업무
- 신사업 기획/기술영업 업무 : ICT사업 기획 및 신규고객 발굴
- 디지털트윈 지능형 관제 사업, 실시간 XR훈련 사업, 메타버스 서비스 사업 등
- 국내 고객(공공기관, 민간 등) 및 해외 파트너 발굴
- 공공/국방 SI, NI 제안사업 업무 : 제안서 작성 주관 및 사업 관리 등
- 핵심 사업 수주/홍보 및 마케팅 업무 : 제안서 작성 주관, 마케팅 활동
- 국내 및 해외 사업 제안서 작성, 전시회 기획/참가, 고객 PPT 등

복지 및 혜택

- 유망산업
- 주차지원
- 51~300명
- 설립10년이상
- 연봉 업계평균이상
- 인원 급성장

우대사항

- 우대사항
- PT 능력 우수자
- 제안서 작성 능력 우수자
- 공공분야(지자체 포함)/국방분야 사업관리 및 영업 경험자(인적 인프라 보유 등)
- ICT 관련학과 또는 SW 관련학과 전공자
- 네트워크 장비/보안 솔루션/영상감시 등 사업 및 영업 경험자
- 글로벌 역량 보유자(역량 수준에 따라 차우 우대)
- 글로벌 학습자료/정보를 이해하고 업무에 활용할 수 있는 능력
- 해외 업무 진행 시 원활한 영어(또는 기타 외국어) 의사소통이 가능한 수준
- * 면접 과정에서 역량 검증 예정
- * 공인 외국어 성적표 등 글로벌 역량 관련 서류 사전 제출 시 서류전형 우대

연봉

회사내규에 따름

지원하기

목록보기

- 공고 상세 정보 제공
- 토글 방식으로 스크랩 가능, 아이콘 변경(로그인한 사용자만 지원 가능)
- 지원하기를 누를 경우 이력서 선택 페이지(/mypage/choice_list/{id})로 이동, 이미 지원한 공고라면 중복 지원을 막고 알림(로그인한 사용자만 지원 가능)

🔖

지원하기

목록보기

🔖

지원하기

목록보기

이력서 선택

☒ 풀스택 개발자

☐ 자바 개발자

☐ 빅데이터 분석가

☐ 자소서1

☐ 자소서2

제출하기

취소

1

2

>

localhost 내용:
[풀스택 개발자] 이력서를 제출하시겠습니까?

확인

취소

4. 프로젝트 수행 경과

사용자 부분

채용공고 리스트 → 채용공고 상세(스크랩 / 지원하기) → 이력서 제출

✓ 채용공고 상세

```
@GetMapping("/empl_detail/{id}") no usages  ⚙ tjoeunJava +1
public String emplDetailPage(
    @PathVariable("id") Long id,
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(required = false) String prevUrl,
    @RequestParam(required = false) String sortOrder,
    @RequestParam(required = false) String title,
    @RequestParam(required = false) String content,
    @RequestParam(required = false) String region,
    @RequestParam(required = false) String company,
    @RequestParam(required = false) String startDate,
    @RequestParam(required = false) String endDate,
    Model model,
    Principal principal) {

    RecruitmentDTO dto = emplService.getRecruitmentDetail(id);
    if (dto == null) {
        return "/main";
    }

    boolean isFavorited = false;
    if (principal != null) {
        String userEmail = principal.getName();
        isFavorited = myPageService.isFavorited(userEmail, id);
    }

    model.addAttribute( attributeName: "page", page);
    model.addAttribute( attributeName: "job", dto);
    model.addAttribute( attributeName: "prevUrl", prevUrl != null ? prevUrl : "/empl/empl_main");
    model.addAttribute( attributeName: "isFavorited", isFavorited);
    model.addAttribute( attributeName: "sortOrder", sortOrder);
    model.addAttribute( attributeName: "title", title);
    model.addAttribute( attributeName: "content", content);
    model.addAttribute( attributeName: "region", region);
    model.addAttribute( attributeName: "company", company);
    model.addAttribute( attributeName: "startDate", startDate);
    model.addAttribute( attributeName: "endDate", endDate);

    return "empl/empl_detail";
}
```

4. 프로젝트 수행 경과

사용자 부분

채용공고 리스트 → 채용공고 상세(스크랩 / 지원하기) → **이력서 제출**

✓ 지원하기

이력서 선택

☒ 풀스택 개발자

☐ 자바 개발자

☐ 빅데이터 분석가

☐ 자소서1

☐ 자소서2

제출하기

취소

1 2 >

localhost 내용:

[풀스택 개발자] 이력서를 제출하시겠습니까?

확인

취소

- 단일 이력서 선택만 가능 (radio 버튼)
- 이미 지원한 공고인 경우 중복 지원 차단 후 알림
- 제출 전 유효성 검사 및 확인창 출력
- 채용공고 ID(recruitmentIdx) 전달
- 이력서 제출 요청 처리 (POST)

4. 프로젝트 수행 경과

사용자 부분

채용공고 리스트 → 채용공고 상세(스크랩 / 지원하기) → **이력서 제출**

✓ 지원하기

```
@GetMapping("/choice_list/{recruitmentIdx}") no usages
public String getResumeList(@PathVariable Long recruitmentIdx,
                             Model model,
                             @AuthenticationPrincipal UserDetails userDetails,
                             @RequestParam(defaultValue = "1") int page,
                             @PageableDefault(size = 5) Pageable pageable) {

    String email = userDetails.getUsername();
    System.out.println("로그인된 이메일: " + email);

    Users user = userService.findByUserEmail(email);
    System.out.println("조회된 userId: " + user.getUserId());

    Long userId = user.getUserId().longValue();
    List<Resume> resumes = myPageService.getResumesByUserId(userId);
    System.out.println("불러온 이력서 수: " + resumes.size());

    Pageable correctedPageable = PageRequest.of(pageNumber: page - 1, pageable.getPageSize(), pageable.getSort());
    Page<Resume> resumePage = myPageService.getPagedResumesByUserId(userId, correctedPageable);

    PaginationUtil.setPaging(model, resumePage, url: "/mypage/choice_list/" + recruitmentIdx);

    System.out.println("총 페이지 수: " + resumePage.getTotalPages());
    System.out.println("현재 페이지 데이터 수: " + resumePage.getContent().size());

    model.addAttribute(attributeName: "resumeList", resumePage.getContent());
    model.addAttribute(attributeName: "jobPage", resumePage);
    model.addAttribute(attributeName: "recruitmentIdx", recruitmentIdx);

    return "mypage/choice_list";
}
```


4. 프로젝트 수행 경과

사용자 부분

지원 현황 → 이력서 확인 / 채용 공고 확인

✓ 지원 현황

마이페이지

회원정보수정

이력서관리

지원현황

스크랩

공공기술영업
네비웍스

진행중
지원 이력서

해외영업 글로벌 세일즈 담당자 (복미) / 신입
제이디코프

진행중
지원 이력서

해외영업 글로벌 세일즈 담당자 (복미) / 3~5년
제이디코프

불합격
지원 이력서

컨테이너 용접사 직원 (경력 무관)
컨테이너테크닉큐서비스㈜

합격
지원 이력서

<핵심 기능>

- 사용자가 지원한 채용공고 목록 표시
- 공고명 클릭 → 채용공고 상세 페이지(empl_detail)
- 지원 이력서 클릭 → 지원 당시 제출한 이력서의 읽기 전용 상세 페이지로 이동
- 각 이력에 대한 지원 상태(status) 표시

4. 프로젝트 수행 경과

사용자 부분

지원 현황 → 이력서 확인 / 채용 공고 확인

✓ 지원 현황

```
@GetMapping("/apply_status") no usages
public String applyStatus(@RequestParam(defaultValue = "1") int page,
                          @RequestParam(required = false) String customSort,
                          @PageableDefault(size = 10) Pageable pageable,
                          Model model,
                          Principal principal) {

    String email = principal.getName();

    Pageable correctedPageable = PageRequest.of(
        pageNumber: page - 1,
        pageable.getPageSize(),
        Sort.by(Sort.Direction.DESC, ...properties: "apply")
    );
    Page<ApplyHistoryDTO> historyPage = myPageService.getPagedApplyHistories(email, correctedPageable);
    PaginationUtil.setPaging(model, historyPage, url: "/mypage/apply_status");
    model.addAttribute(attributeName: "historyList", historyPage.getContent());
    model.addAttribute(attributeName: "jobPage", historyPage);

    return "mypage/apply_status";
}
```

4. 프로젝트 수행 경과

사용자 부분

지원 현황 → 이력서 확인 / 채용 공고 확인

✓ 지원한 이력서 확인

마이페이지 > 지원현황 > 지원 이력서

제목	플스텍 개발자
이름 사용자 이메일 user@naver.com 생년월일 2025-07-09 주소 서울특별시 00구 000로 00길 00 000동 000호 전화번호 010-1234-1234 학력 00대학 졸업 자격증 및 스킨 00000가시 자격증 경력 00000가시 3개월 인턴 경험 ###로비비즈 2개월 수상이력 졸업작품 최우수	
본인을 한 단어로 표현한다면, 그리고 그 이유는 무엇인가요?	저를 한 단어로 표현한다면 '성실함'입니다. 저는 주어진 일에 책임감을 갖고 끝까지 완수하려는 태도를 항상 중요하게 여깁니다. 대학교 팀 프로젝트에서는 맡은 역할 외에도 전서 일정 관리와 자료 정리를 도맡으며 팀의 완성도를 높였습니다. 덕분에 팀은 학기 최우수 프로젝트로 선정될 수 있었고, 이 경험을 통해 성실함이 단순한 노력 이상의 가치를 가진다는 것을 배웠습니다. 조직 내에서 신뢰받는 사람이 되는 것이 저의 목표이며, 성실함은 이를 이루는 핵심 역량이라 믿습니다.
가장 어려웠던 경험과 그것을 극복한 방법은 무엇인가요?	대학교 3학년 때 전공 수업에서 난이도가 높은 데이터 분석 프로젝트를 맡았을 때가 가장 어려웠던 경험이었습니다. 팀원 모두 Python 사용 경험이 부족했고, 처음에는 분석 방향도 잡기 어려웠습니다. 저는 팀을 대표해 직접 교수님과 상담하고, 온라인 강의를 찾아 공부하면서 기초부터 다져갔습니다. 이후 팀원들과 자료를 공유하고 간단한 코드부터 함께 실습하며 협업을 이어갔습니다. 결과적으로 프로젝트를 성공적으로 마무리했고, 발표에서도 높은 평가를 받았습니다. 이 경험을 통해 어려움은 회피보다 '구체적인 행동'으로 풀어가야 한다는 교훈을 얻었습니다.
질문과 답변을 변경	기존에 있던 질문과 답변을 변경해보겠습니다.
추가적인 질문 사항	새로운 질문과 답변을 추가해보겠습니다.
포트폴리오 링크	https://github.com/RhoAias12/Project_tjeoun.git

목록보기

✓ 채용 공고 확인

신입

지역

시작일
연도-월-일

공고내용

회사명

마감일
연도-월-일

검색

공공기술영업

네비웍스

상시채용 | 지역: 경기 안양시

고용형태

연접 후 결정

요구사항

자격요건

- 공공/국방 SI, NI 사업분야 경력 5~10년(과/차장급)
- ICT 사업기획/영업/제안/개발 업무 경험이 있으신 분
- 공공/국방 사업 제안서 작성 경력이 있으신 분
- 정보통신 분야 자격증을 보유하신 분

* 공인 외국어 성적표 등 글로벌 역량 관련 서류 사전 제출 시 서류전형 우대

연봉

회사내규에 따름

🔖

지원하기

목록보기

- 이력서 세부 참고
- 채용 공고 세부 참고

4. 프로젝트 수행 경과

사용자 부분

스크랩 → 채용공고 확인 / 지원하기

✓ 스크랩

마이페이지

[회원정보수정](#)[이력서관리](#)[지원현황](#)[스크랩](#)

공공기술영업

네비웍스

상시채용

지원하기

[이지케어] 재가요양기관 ERP 영업 담당

기웅정보통신

마감일
2025.07.12

지원하기

[공공기관] 학교안전공제중앙회 재무회계 보조

유니에스 기업정보

마감일
2025.08.09

지원하기

컨테이너 용접사 직원 (경력 무관)

컨테이너테크닉큐서비스㈜

마감일
2025.07.24

지원하기

- 사용자가 스크랩한 채용공고 목록 조회
- 공고명 클릭 시 채용 상세 페이지로 이동
- 스크랩 삭제 버튼
- 지원 이력서 클릭 → 지원 당시 제출한 이력서의 읽기 전용 상세 페이지로 이동
- 지원 중복 방지 (지원하기 버튼 클릭 시 /mypage/check_applied/{recruitmentId} 요청을 먼저 보내 응답 결과에 따라 분기)
- 페이지네이션

4. 프로젝트 수행 경과

사용자 부분

스크랩 → 채용공고 확인 / 지원하기

✓ 스크랩

```
@GetMapping("/scrap") no usages
public String scrapPage(@RequestParam(defaultValue = "1") int page,
                        @RequestParam(required = false) String customSort,
                        @PageableDefault(size = 8) Pageable pageable,
                        Model model,
                        Principal principal) {

    String email = principal.getName();
    Pageable correctedPageable = PageRequest.of(
        Math.max(page - 1, 0),
        pageable.getPageSize(),
        Sort.by(Sort.Direction.DESC, ...properties: "apply")
    );

    Page<FavoriteDTO> jobPage = myPageService.getPagedFavorites(email, correctedPageable);

    int totalPages = jobPage.getTotalPages();
    if (totalPages > 0 && page > totalPages) {
        return "redirect:/mypage/scrap?page=" + totalPages;
    }
    model.addAttribute(attributeName: "favorites", jobPage.getContent());
    model.addAttribute(attributeName: "jobPage", jobPage);
    model.addAttribute(attributeName: "customSort", customSort);
    PaginationUtil.setPaging(model, jobPage, url: "/mypage/scrap");

    return "mypage/scrap";
}
```

```
@DeleteMapping("/scrap") no usages
@ResponseBody
@Transactional
public ResponseEntity<> deleteScrap(@RequestBody FavoriteDTO favoriteDTO,
                                    @RequestParam(required = false) Integer page,
                                    @RequestParam(required = false, defaultValue = "8") Integer size) {

    myPageService.deleteScrap(favoriteDTO);
    Users user = userRepository.findById(favoriteDTO.getUserIdx())
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    Pageable pageable = PageRequest.of(Math.max(page - 1, 0), size); // page는 0-based
    Page<Favorite> resultPage = favoriteRepository.findByUser(user, pageable);

    int currentPage = Math.max(page != null ? page : 1, 1);
    int newPage = resultPage.getTotalElements() == 0 && currentPage > 1
        ? currentPage - 1
        : currentPage;

    return ResponseEntity.ok().body(Map.of(k1: "newPage", newPage));
}
```


4. 프로젝트 수행 경과

사용자 부분

스크랩 → 채용공고 확인 / 지원하기

✓ 채용공고 확인

신입

지역

지역

연도-월-일

공고내용

회사명

연도-월-일

연도-월-일

검색

공공기술영업

네비웍스

상시채용 | 지역: 경기 안양시

Naviworks

고용형태

면접 후 결정

요구사항

자격요건

- 공공/국방 SI, NI 사업분야 경력 5~10년(과/차장급)
- ICT 사업기획/영업/제안/개발 업무 경험이 있으신 분
- 공공/국방 사업 제안서 작성 경력이 있으신 분
- 정보통신 분야 자격증을 보유하신 분

연봉

회사내규에 따름

지원하기

목록보기

✓ 지원하기

마이페이지

localhost 내용:
이미 지원한 공고입니다. 중복 지원은 불가능합니다.

확인

마이페이지

회원정보수정

이력서관리

지원현황

스크랩

공공기술영업

네비웍스

상시채용

지원하기

- 채용 공고 세부 참고
- 공고 지원 참고

4. 프로젝트 수행 경과

관리자 부분

회원 관리 리스트/삭제 → 회원 정보 열람 / 수정

✓ 회원 관리 검색/정렬/삭제

회원관리

검색

가입일 : 최신순

가입일 : 최신순
가입일 : 오래된순
이메일순 : 오름차순
이메일순 : 내림차순
닉네임순 : 오름차순
닉네임순 : 내림차순

user@naver.com 사용자2222	
son18@example.com sonnick18	삭제
lim17@example.com limnick17	삭제

- 전체 회원 리스트 조회
- 이메일 닉네임 검색
- sortBy 파라미터 기반 정렬 구현 (latest, oldest, email, nickname 선택 시 자동 반영)
- 회원 클릭 시 수정 페이지로 이동
- 회원 삭제 기능
- 삭제 후 남은 회원 수에 따라 마지막 페이지로 자동 조정
- 엘라스틱 검색 구현 참고

4. 프로젝트 수행 경과

관리자 부분

회원 관리 리스트/삭제 → 회원 정보 열람 / 수정

✓ 회원 관리 리스트 확인

- 회원 목록을 검색 조건 및 정렬 기준에 따라 DB 또는 Elasticsearch에서 조회하여 화면에 출력

```
@GetMapping("/member_list") no usages 呂 green4141 +2 *
public String memberList(
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(defaultValue = "latest") String sortBy,
    @RequestParam(required = false) String email,
    @RequestParam(required = false) String nickname,
    @PageableDefault(size = 10) Pageable pageable,
    Model model) throws IOException {

    Page<UserListDto> userPage;
    if ((email != null && !email.isBlank()) || (nickname != null && !nickname.isBlank())) {
        userPage = adminService.getUsersBySearchFromES(email, nickname, sortBy, page, pageable.getPageSize());
    } else {
        userPage = adminService.getPagedUsers(page, pageable.getPageSize(), sortBy);
    }

    StringBuilder url = new StringBuilder("/admin/member_list?sortBy=" + sortBy);
    if (email != null && !email.isBlank()) url.append("&email=").append(email);
    if (nickname != null && !nickname.isBlank()) url.append("&nickname=").append(nickname);
    PaginationUtil.setPaging(model, userPage, url.toString());
    model.addAttribute(attributeName: "userList", userPage.getContent());
    model.addAttribute(attributeName: "userPage", userPage);
    model.addAttribute(attributeName: "sortBy", sortBy);
    model.addAttribute(attributeName: "page", page);
    model.addAttribute(attributeName: "email", email);
    model.addAttribute(attributeName: "nickname", nickname);

    return "admin/member_list";
}
```

✓ 회원 삭제

- 회원 정보를 삭제한 뒤, 검색 조건을 유지한 상태로 목록 페이지를 갱신하여 리디렉션

```
@PostMapping("/member_list/delete") no usages 呂 RhoAias12 +2 *
public String deleteMember(
    @RequestParam("userIdx") Integer userIdx,
    @RequestParam("page") int page,
    @RequestParam("sortBy") String sortBy,
    @RequestParam(required = false) String email,
    @RequestParam(required = false) String nickname
) throws IOException {
    userService.deleteById(userIdx);
    int size = 10;
    Page<UserListDto> userPage;
    if ((email != null && !email.isBlank()) || (nickname != null && !nickname.isBlank())) {
        userPage = adminService.getUsersBySearchFromES(email, nickname, sortBy, page, size);
    } else {
        userPage = adminService.getPagedUsers(page, size, sortBy);
    }
    if (userPage.isEmpty() && page > 1) {
        page--;
    }
    StringBuilder redirectUrl = new StringBuilder("/admin/member_list?page=" + page + "&sortBy=" + sortBy);
    if (email != null) redirectUrl.append("&email=").append(URLEncoder.encode(email, StandardCharsets.UTF_8));
    if (nickname != null) redirectUrl.append("&nickname=").append(URLEncoder.encode(nickname, StandardCharsets.UTF_8));

    return "redirect:" + redirectUrl;
}
```

4. 프로젝트 수행 경과

관리자 부분

회원 관리 리스트/삭제 → **회원 정보 열람 / 수정**

✓ 회원 정보 열람

회원정보수정 (관리자)

이름

사용자

이메일

user@naver.com

닉네임

사용자2222

중복확인

비밀번호

비밀번호 (변경 시 입력)

비밀번호 확인

비밀번호 확인

생년월일

2025-07-09

권한

☒ USER
☐ ADMIN

수정

목록

- 선택한 회원의 이름, 이메일, 닉네임, 생년월일, 권한 정보 조회
- 닉네임 수정 + 중복 확인, 비밀번호 변경, 권한 변경 기능
- "목록" 버튼 클릭 시 page, sortBy 유지하여 리스트 복귀
- 사용자 회원 정보 수정에서 권한 정보를 추가한 동일한 방식

```
if (dto.getUserRole() != null) {  
    user.setUserRole(dto.getUserRole());  
}  
  
userRepository.save(user);
```

4. 프로젝트 수행 경과

관리자 부분

공고 관리 리스트 / 삭제 → 공고 수정

✓ 공고 관리 리스트

공고관리

공고이름

지역

시작일

연도-월-일

공고내용

회사명

마감일

연도-월-일

검색

전체

전체

마감일: 내림차순

마감일: 오름차순

플라츠

지원모집

삭제

[차고지서초] 현대자동차 임원 수행기사 채용 (7/11금 마감)

(주)동양이엠에스 기업정보

지원모집

삭제

(일본/나리타공항-WRSJ) 공항지상직 STAFF 모집

(주)위더스토탈에어포트서비스

지원모집

삭제

지원모집

- 채용 공고 목록 조회 및 페이지네이션
- 공고명, 회사명, 마감일, 상태(지원모집/지원마감) 출력
- 공고 검색 기능 (공고명, 내용, 지역, 회사명, 기간 필터)
- 마감일 정렬 (전체 / 내림차순 / 오름차순)
- 공고 삭제 기능 (삭제 전 확인창)
- 공고 카드 클릭 시 상세 수정 페이지로 이동

4. 프로젝트 수행 경과

관리자 부분

공고 관리 리스트 / 삭제 → 공고 수정

✓ 공고 관리 리스트

신입	공고내용
지역	회사명
시작일 연도-월-일	마감일 연도-월-일
검색	
마감일: 오름차순	
(주)디에스피 온라인마케팅팀(신입/경력) 정규직 채용 (주)디에스피	
지원마감 삭제	
AX 파트너, 허들러스에서 그로스 컨설턴트를 채용합니다.(신입 + 경력) 허들러스	
지원마감 삭제	
AX 파트너, 허들러스에서 Growth Manager / CSM을 채용합니다. (신입,경력) 허들러스	
지원모집 삭제	

✓ 공고 삭제

localhost 내용:
정말 삭제하시겠습니까?

확인 취소

공고관리

신입	공고내용
지역	회사명
시작일 연도-월-일	마감일 연도-월-일
검색	
마감일: 오름차순	
AX 파트너, 허들러스에서 그로스 컨설턴트를 채용합니다.(신입 + 경력) 허들러스	
지원마감 삭제	
AX 파트너, 허들러스에서 Growth Manager / CSM을 채용합니다. (신입,경력) 허들러스	
지원모집 삭제	

4. 프로젝트 수행 경과

관리자 부분

공고 관리 리스트 / 삭제 → 공고 수정

✓ 공고 관리 리스트

AdminService

```
public Page<RecruitmentDTO> getFilteredRecruitmentsByEs( 1 usage  ⚡ green4141 +1
    String title,
    String content,
    String region,
    String company,
    String startDate,
    String endDate,
    String deadlineSort,
    int page,
    int size) throws IOException {

    Page<RecruitmentDocument> esPage = recruitmentSearchService.searchJobs(
        title, content, region, company, startDate, endDate, deadlineSort, page, size);

    List<RecruitmentDTO> dtoList = esPage.getContent().stream() Stream<RecruitmentDocument>
        .map( RecruitmentDocument doc -> RecruitmentDTO.builder()
            .recruitmentIdx(doc.getRecruitmentIdx())
            .title(doc.getTitle())
            .company(doc.getCompany())
            .deadline(doc.getDeadline() != null ? LocalDateTime.parse(doc.getDeadline(), formatter) : null)
            .scrapCount(doc.getScrapCount() != null ? doc.getScrapCount() : 0)
            .logoUrl(doc.getLogoUrl())
            .build()) Stream<RecruitmentDTO>
        .toList();

    return new PageImpl<>(dtoList, esPage.getPageable(), esPage.getTotalElements());
}
```

AdminController

```
@GetMapping("/recruit_list") no usages  ⚡ tjoeunJava +2
public String recruitList(
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(defaultValue = "deadline_all") String deadlineSort,
    @RequestParam(required = false) String title,
    @RequestParam(required = false) String content,
    @RequestParam(required = false) String region,
    @RequestParam(required = false) String company,
    @RequestParam(required = false) String startDate,
    @RequestParam(required = false) String endDate,
    @PageableDefault(size = 10) Pageable pageable,
    Model model) throws IOException {

    // ES 기반 검색 호출
    Page<RecruitmentDTO> recruitmentPage = adminService.getFilteredRecruitmentsByEs(
        title, content, region, company, startDate, endDate, deadlineSort, page, pageable.getPageSize()
    );

    StringBuilder urlWithParams = new StringBuilder("/admin/recruit_list?deadlineSort=" + deadlineSort);
    if (title != null && !title.isBlank()) urlWithParams.append("&title=").append(title);
    if (content != null && !content.isBlank()) urlWithParams.append("&content=").append(content);
    if (region != null && !region.isBlank()) urlWithParams.append("&region=").append(region);
    if (company != null && !company.isBlank()) urlWithParams.append("&company=").append(company);
    if (startDate != null && !startDate.isBlank()) urlWithParams.append("&startDate=").append(startDate);
    if (endDate != null && !endDate.isBlank()) urlWithParams.append("&endDate=").append(endDate);

    PaginationUtil.setPaging(model, recruitmentPage, urlWithParams.toString());

    model.addAttribute( attributeName: "recruitmentList", recruitmentPage.getContent());
    model.addAttribute( attributeName: "recruitmentPage", recruitmentPage);
    model.addAttribute( attributeName: "deadlineSort", deadlineSort);
    model.addAttribute( attributeName: "page", page);

    model.addAttribute( attributeName: "title", title);
    model.addAttribute( attributeName: "content", content);
    model.addAttribute( attributeName: "region", region);
    model.addAttribute( attributeName: "company", company);
    model.addAttribute( attributeName: "startDate", startDate);
    model.addAttribute( attributeName: "endDate", endDate);

    return "admin/recruit_list";
}
```


4. 프로젝트 수행 경과

관리자 부분

공고 관리 리스트 / 삭제 → 공고 수정

✓ 공고 삭제

```
public void deleteRecruitmentById(Long recruitmentIdx) {  👤 green4141 +1
    recruitmentRepository.deleteById(recruitmentIdx);
    try {
        recruitmentSearchService.deleteById(recruitmentIdx);

        List<ApplyHistoryDocument> relatedHistories = applyHistorySearchRepository.findByRecruitmentId(recruitmentIdx);

        relatedHistories.forEach( ApplyHistoryDocument doc -> {
            applyHistorySearchRepository.deleteById(doc.getApplyHistoryId());
        });
    } catch (IOException e) {
        e.printStackTrace();
    }
}

public int countRecruitments() { return (int) recruitmentRepository.count(); }
```

AdminService

AdminController

```
@PostMapping("/recruit_delete")  no usages  👤 tjoeunJava
public String deleteRecruit(@RequestParam("recruitmentIdx") Long recruitmentIdx,
                             @RequestParam(defaultValue = "1") int page,
                             @RequestParam(defaultValue = "deadline_all") String deadlineSort) {

    adminService.deleteRecruitmentById(recruitmentIdx);

    int totalCount = adminService.countRecruitments();
    int pageSize = 10;

    int totalPages = (int) Math.ceil((double) totalCount / pageSize);

    int finalPage = Math.min(page, Math.max(totalPages, 1));

    return "redirect:/admin/recruit_list?page=" + finalPage
        + "&deadlineSort=" + deadlineSort;
}
```

4. 프로젝트 수행 경과

관리자 부분

공고 관리 리스트 / 삭제 → **공고 수정**

✓ 공고 수정

공고 관리

AX 파트너, 허들러스에서 그로스 컨설턴트를 채용합니다.(신입 + 경력)

파일 선택 선택...

허들러스

마감일: 2025-07-10 ☐ 지역: 서울

고용형태

정규직

요구사항

- 기존 마케팅 개론 및 디지털 마케팅 분야의 관심자/전공자

- 초, 중급 수준의 영어 실력 (자료, 영상, 글을 어느 정도 읽고 이해할 수 있을 만한 능력)

- Google Drive, Sheet, Notion, Slack 등 각종 문서 및 소통 툴에 익숙한 사람

- 관리자용 공고 수정 폼 제공
- 공고 기본 정보 및 상세 내용 수정
- 회사 로고 이미지 업로드 및 미리보기
- 수정 전 확인 알림
- 수정 후 목록 페이지로 이동

우대사항

- **(우대사항) 커뮤니케이션 능력이 좋아 다른 부서 또는 클라이언트와 PM으로서의 소통 역량이 출중한 사람**

- **(우대사항) 이벤트 텍소노미를 설계해본 경험이 있는 사람**

- **(우대사항) 비즈니스 수준의 영어 능력을 가지고 있는 사람**

- **(우대사항) GPT 등 AI 관련 툴에 익숙한 사람**

- **(우대사항) 페이스북/네이버/구글 등 광고 매체 운영 경험이 있는 사람**

- **(우대사항) 간단한 웹/앱 또는 프로그래밍 지식이 있는 사람**

연봉

3300만원

목록보기

수정하기

4. 프로젝트 수행 경과

관리자 부분

공고 관리 리스트 / 삭제 → **공고 수정**

✓ 공고 수정

AdminService

```
@Transactional 1 usage green4141 +1
public void updateRecruitment(RecruitmentDTO dto) {
    Recruitment entity = recruitmentRepository.findById(dto.getRecruitmentIdx())
        .orElseThrow(() -> new IllegalArgumentException("해당 공고가 존재하지 않습니다."));

    entity.setTitle(dto.getTitle());
    entity.setCompany(dto.getCompany());
    entity.setDeadline(dto.getDeadline());
    entity.setQualifications(dto.getQualifications());
    entity.setLogoUrl(dto.getLogoUrl());
    entity.setResponsibilities(dto.getResponsibilities());
    entity.setPreferred(dto.getPreferred());
    entity.setBenefits(dto.getBenefits());
    entity.setLocation(dto.getLocation());
    entity.setSalary(dto.getSalary());
    entity.setEmploymentType(dto.getEmploymentType());

    recruitmentRepository.save(entity);

    RecruitmentDocument doc = RecruitmentDocument.builder()
        .recruitmentIdx(entity.getRecruitmentIdx())
        .title(entity.getTitle())
        .company(entity.getCompany())
        .deadline(entity.getDeadline() != null ? entity.getDeadline().format(formatter) : null)
        .qualifications(entity.getQualifications())
        .logoUrl(entity.getLogoUrl())
        .responsibilities(entity.getResponsibilities())
        .preferred(entity.getPreferred())
        .benefits(entity.getBenefits())
        .location(entity.getLocation())
        .salary(entity.getSalary())
        .employmentType(entity.getEmploymentType())
        .build();

    try {
        recruitmentSearchService.saveOrUpdate(doc);
        List<ApplyHistory> histories = applyHistoryRepository.findByRecruitment(entity);
        for (ApplyHistory history : histories) {
            applyHistorySearchService.save(history);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

```
@GetMapping("/recruit_modify/{recruitmentIdx}") no usages green4141 +2
public String recruitModify(@PathVariable("recruitmentIdx") Long recruitmentIdx,
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(defaultValue = "deadline_all") String deadlineSort,
    Model model) {

    RecruitmentDTO dto = recruitmentService.getPostById(recruitmentIdx);
    model.addAttribute(attributeName: "recruit", dto);
    model.addAttribute(attributeName: "page", page);
    model.addAttribute(attributeName: "deadlineSort", deadlineSort);
    return "admin/recruit_modify";
}
```

AdminController

```
@PostMapping("/recruit_modify") no usages green4141 *
public String modifyRecruit(@ModelAttribute RecruitmentDTO dto,
    @RequestParam("logo") MultipartFile logoFile,
    @RequestParam("page") int page,
    @RequestParam("deadlineSort") String deadlineSort) throws IOException {

    if (!logoFile.isEmpty()) {
        String filename = dto.getCompany() + ".jpg";
        String logoPath = "/logos/" + filename;
        File saveFile = new File(pathname: uploadRoot + logoPath);

        saveFile.getParentFile().mkdirs();
        logoFile.transferTo(saveFile);

        dto.setLogoUrl("/uploads" + logoPath);
    }

    adminService.updateRecruitment(dto);

    return "redirect:/admin/recruit_modify/" + dto.getRecruitmentIdx()
        + "?page=" + page + "&deadlineSort=" + deadlineSort;
}
```

4. 프로젝트 수행 경과

관리자 부분

지원 관리 리스트 → 지원 이력서 확인 → 이력서 상태 수정

✓ 지원 관리 리스트

지원관리

공고이름

지역

이메일

시작일

공고내용

회사명

닉네임

마감일

연도-월-일

-

연도-월-일

검색

전체

상태 전체

[공공기관] 학교안전공제중앙회 재무회계 보조

유니에스 기업정보

user@naver.com | 사용자2222

진행중

[천안시-직전연봉 130프로/리콜비450만] 지점장,사업단장,본부장 채용

에이아이프리미어파트너스(주)

user@naver.com | 사용자2222

진행중

컨테이너 용접사 직원 (경력 무관)

컨테이너테크닉큐서비스(주)

user@naver.com | 사용자2222

합격

<핵심 기능>

- 공고명 클릭 시 해당 지원 상세 페이지로 이동 (페이지 번호, 정렬 상태 유지)
- 검색 필터: 공고이름, 공고내용, 지역, 회사명, 이메일, 닉네임, 지원 날짜 범위 (시작일 ~ 마감일)
- 정렬 옵션: 전체, 지원일 최신/오래된순, 마감일 최신/오래된순
- 상태 필터: 전체, 진행중(SUBMITTED), 합격(PASS), 불합격(FAIL)
- 엘라스틱 검색 구현 참고

4. 프로젝트 수행 경과

관리자 부분

지원 관리 리스트 → 지원 이력서 확인 → 이력서 상태 수정

✓ 지원 관리 리스트

AdminService

```
public Page<ApplyHistoryDTO> getPagedApplyHistory(int page, int size, String sortOption) {
    Sort sort = Sort.unsorted();

    switch (sortOption) {
        case "apply_latest":
            sort = Sort.by(Sort.Direction.DESC, ...properties: "apply");
            break;
        case "apply_oldest":
            sort = Sort.by(Sort.Direction.ASC, ...properties: "apply");
            break;
        case "deadline_latest":
            sort = Sort.by(Sort.Direction.DESC, ...properties: "recruitment.deadline");
            break;
        case "deadline_oldest":
            sort = Sort.by(Sort.Direction.ASC, ...properties: "recruitment.deadline");
            break;
    }

    Pageable pageable = PageRequest.of(page, size, sort);
    Page<ApplyHistory> applyHistoryPage = applyHistoryRepository.findAll(pageable);

    return applyHistoryPage.map( ApplyHistory applyHistory -> ApplyHistoryDTO.builder()
        .applyHistoryId(applyHistory.getOptionalIdx())
        .statusDisplay(applyHistory.getStatus().getDisplay())
        .recruitmentTitle(applyHistory.getRecruitment().getTitle())
        .recruitmentCompany(applyHistory.getRecruitment().getCompany())
        .recruitmentId(applyHistory.getRecruitment().getRecruitmentIdx())
        .userEmail(applyHistory.getUser().getUserEmail())
        .userNickname(applyHistory.getUser().getUserNickname())
        .build());
}
```

```
@Transactional(readOnly = true) 2 usages green4141
public Page<ApplyHistoryDTO> searchApplyHistoriesFromES(
    String recruitmentTitle,
    String recruitmentCompany,
    String userEmail,
    String userNickname,
    String status,
    String startDate,
    String endDate,
    int page,
    int size,
    String sortOption
) throws IOException {
```

```
    Page<ApplyHistoryDocument> esPage = applyHistorySearchService.searchApplyHistories(
        recruitmentTitle, recruitmentCompany, userEmail, userNickname, status, startDate, endDate, page, size, sortOption
    );
```

```
    List<ApplyHistoryDTO> dtoList = esPage.getContent().stream() Stream<ApplyHistoryDocument>
        .map( ApplyHistoryDocument doc -> ApplyHistoryDTO.builder()
            .applyHistoryId(doc.getApplyHistoryId())
            .statusDisplay(getStatusDisplayName(doc.getStatus()))
            .status(doc.getStatus())
            .recruitmentTitle(doc.getRecruitmentTitle())
            .recruitmentCompany(doc.getRecruitmentCompany())
            .userEmail(doc.getUserEmail())
            .userNickname(doc.getUserNickname())
            .build()) Stream<ApplyHistoryDTO>
        .collect(Collectors.toList());
```

```
    return new PageImpl<>(dtoList, esPage.getPageable(), esPage.getTotalElements());
}
```

```
public String getStatusDisplayName(String status) {
    return switch (status) {
        case "SUBMITTED" -> "진행중";
        case "FINALIZED" -> "합격";
        case "REJECTED" -> "불합격";
        default -> "알 수 없음";
    };
}
```

4. 프로젝트 수행 경과

관리자 부분

지원 관리 리스트 → 지원 이력서 확인 → 이력서 상태 수정

✓ 지원 관리 리스트

AdminController

```
@GetMapping("/apply_list") no usages green4141 +1
public String applyListPage(
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "all") String sortOption,
    @RequestParam(required = false) String title,
    @RequestParam(required = false) String company,
    @RequestParam(required = false) String email,
    @RequestParam(required = false) String nickname,
    @RequestParam(required = false) String status,
    @RequestParam(required = false) String startDate,
    @RequestParam(required = false) String endDate,
    Model model
) throws IOException {
    System.out.println("status: " + status);

    int pageIndex = (page < 1) ? 0 : (page - 1);

    Page<ApplyHistoryDTO> applyPage;

    if ((title != null && !title.isBlank()) ||
        (company != null && !company.isBlank()) ||
        (email != null && !email.isBlank()) ||
        (nickname != null && !nickname.isBlank()) ||
        (status != null && !status.isBlank()) ||
        (startDate != null && !startDate.isBlank()) ||
        (endDate != null && !endDate.isBlank())) {

        applyPage = adminService.searchApplyHistoriesFromES(
            title, company, email, nickname, status, startDate, endDate, pageIndex, size, sortOption
        );
    } else {
        applyPage = adminService.getPagedApplyHistory(pageIndex, size, sortOption);
    }
}
```

Elasticsearch + JPA 혼합 방식의 지원관리 리스트

- 검색조건이 있을시에만 ES 사용, 조건이 없으면 JPA로 전체 지원 목록 조회

```
StringBuilder url = new StringBuilder("/admin/apply_list?sortOption=" + sortOption);
if (title != null && !title.isBlank()) url.append("&title=").append(title);
if (company != null && !company.isBlank()) url.append("&company=").append(company);
if (email != null && !email.isBlank()) url.append("&email=").append(email);
if (nickname != null && !nickname.isBlank()) url.append("&nickname=").append(nickname);
if (status != null && !status.isBlank()) url.append("&status=").append(status);
if (startDate != null && !startDate.isBlank()) url.append("&startDate=").append(startDate);
if (endDate != null && !endDate.isBlank()) url.append("&endDate=").append(endDate);

PaginationUtil.setPaging(model, applyPage, url.toString());

model.addAttribute(attributeName: "applyList", applyPage.getContent());
model.addAttribute(attributeName: "applyPage", applyPage);
model.addAttribute(attributeName: "sortOption", sortOption);

model.addAttribute(attributeName: "title", title);
model.addAttribute(attributeName: "company", company);
model.addAttribute(attributeName: "email", email);
model.addAttribute(attributeName: "nickname", nickname);
model.addAttribute(attributeName: "status", status);
model.addAttribute(attributeName: "startDate", startDate);
model.addAttribute(attributeName: "endDate", endDate);

return "admin/apply_list";
}
```


4. 프로젝트 수행 경과

관리자 부분

지원 관리 리스트 → 지원 이력서 확인 → 이력서 상태 수정

✓ 지원 이력서 확인 / 이력서 상태 수정

지원 관리

공공기술영업
네비웍스
마감일: 9999-12-31 지역: 경기 안양시

제목

풀스택 개발자

이름
사용자

이메일
user@naver.com

생년월일
2025-07-09

주소
서울특별시 00구 00로 00길 00 000동 000호

전화번호
010-1234-1234

학력
00대학 졸업

자격증 및 스킬
00000기사 자격증

합격

진행중

합격

불합격

localhost 내용:

합격(으)로 변경하시겠습니까?

확인

취소

공공기술영업
네비웍스
user@naver.com | 사용자2222

합격

<핵심 기능>

- 선택한 지원 내역의 상세 정보(지원자 정보, 이력서 내용, 지원 공고 정보) 읽기 전용 출력
- 지원 상태(진행중, 합격, 불합격) 드롭다운 선택 가능
- 지원 상태 변경 시 확인 팝업 후 폼 제출하여 상태 업데이트

4. 프로젝트 수행 경과

관리자 부분

지원 관리 리스트 → **지원 이력서 확인** → 이력서 상태 수정

✓ 지원 이력서 확인

AdminService

```
@Transactional(readOnly = true) 1 usage 2 tjoeunJava +1
public ApplyDetailDTO getApplyDetailById(Integer applyHistoryId) {
    ApplyHistory applyHistory = applyHistoryRepository.findById(applyHistoryId)
        .orElseThrow(() -> new IllegalArgumentException("해당 지원 기록을 찾을 수 없습니다. id=" + applyHistoryId));

    Recruitment recruitment = applyHistory.getRecruitment();
    ApplyHistoryResume resume = applyHistory.getApplyHistoryResume();
    Users user = applyHistory.getUser();

    List<ResumeContentDTO> resumeContentList = resume.getContents().stream() Stream<ApplyHistoryResumeContent>
        .map( ApplyHistoryResumeContent content -> ResumeContentDTO.builder()
            .question(content.getQuestion())
            .context(content.getContext())
            .build()) Stream<ResumeContentDTO>
        .collect(Collectors.toList());

    return ApplyDetailDTO.builder()
        .recruitmentTitle(recruitment.getTitle())
        .recruitmentCompany(recruitment.getCompany())
        .recruitmentDeadline(LocalDate.from(recruitment.getDeadline()))
        .recruitmentLocation(recruitment.getLocation())
        .recruitmentLogoUrl(recruitment.getLogoUrl())

        .resumeTitle(resume.getTitle())
        .resumeImg(resume.getImg())
        .resumeAddress(resume.getAddress())
        .resumePhoneNum(resume.getPhoneNum())
        .resumeEducation(resume.getEducation())
        .resumeAbility(resume.getAbility())
        .resumeAntecedents(resume.getAntecedents())
        .resumeAwards(resume.getAwards())
        .resumeContext(resume.getContext())

        .userName(user.getUserName())
        .userEmail(user.getUserEmail())
        .userBirth(user.getUserBirth())

        .resumeContentList(resumeContentList)
        .statusDisplay(applyHistory.getStatus().getDisplay())
        .build();
}
```

AdminController

```
@GetMapping("/apply_detail/{applyHistoryId}") no usages 2 green4141 +1
public String applyDetail(@PathVariable Integer applyHistoryId,
    @RequestParam(value = "page", defaultValue = "1") int page,
    @RequestParam(value = "sortOption", defaultValue = "all") String sortOption,
    @RequestParam(required = false) String title,
    @RequestParam(required = false) String company,
    @RequestParam(required = false) String email,
    @RequestParam(required = false) String nickname,
    @RequestParam(required = false) String status,
    @RequestParam(required = false) String startDate,
    @RequestParam(required = false) String endDate,
    Model model) {

    ApplyDetailDTO applyDetail = adminService.getApplyDetailById(applyHistoryId);
    model.addAttribute( attributeName: "applyDetail", applyDetail);
    model.addAttribute( attributeName: "applyHistoryId", applyHistoryId);
    model.addAttribute( attributeName: "page", page);
    model.addAttribute( attributeName: "sortOption", sortOption);
    model.addAttribute( attributeName: "title", title);
    model.addAttribute( attributeName: "company", company);
    model.addAttribute( attributeName: "email", email);
    model.addAttribute( attributeName: "nickname", nickname);
    model.addAttribute( attributeName: "status", status);
    model.addAttribute( attributeName: "startDate", startDate);
    model.addAttribute( attributeName: "endDate", endDate);
    return "admin/apply_detail";
}
```

4. 프로젝트 수행 경과

관리자 부분

지원 관리 리스트 → 지원 이력서 확인 → **이력서 상태 수정**

✓ 이력서 상태 수정

AdminController

```
@PostMapping("/apply_detail/{applyHistoryId}/updateStatus") no usages 2 green4141 +1 *
public String updateApplyStatus(@PathVariable Integer applyHistoryId,
                                @RequestParam("newStatus") String newStatus,
                                @RequestParam(value = "page", defaultValue = "1") int page,
                                @RequestParam(value = "sortOption", defaultValue = "apply_latest") String sortOption,
                                @RequestParam(required = false) String title,
                                @RequestParam(required = false) String company,
                                @RequestParam(required = false) String email,
                                @RequestParam(required = false) String nickname,
                                @RequestParam(required = false) String status,
                                @RequestParam(required = false) String startDate,
                                @RequestParam(required = false) String endDate,
                                RedirectAttributes redirectAttributes) throws IOException {

    adminService.updateApplyStatus(applyHistoryId, newStatus);

    String statusDisplay = adminService.getStatusDisplayName(newStatus);

    int size = 10;
    int pageIndex = page - 1 < 0 ? 0 : page - 1;

    Page<ApplyHistoryDTO> applyPage;

    if ((title != null && !title.isBlank()) ||
        (company != null && !company.isBlank()) ||
        (email != null && !email.isBlank()) ||
        (nickname != null && !nickname.isBlank()) ||
        (status != null && !status.isBlank()) ||
        (startDate != null && !startDate.isBlank()) ||
        (endDate != null && !endDate.isBlank())) {
        applyPage = adminService.searchApplyHistoriesFromES(
            title, company, email, nickname, status, startDate, endDate, pageIndex, size, sortOption
        );
    } else {
        applyPage = adminService.getPagedApplyHistory(pageIndex, size, sortOption);
    }
}
```

```
@Transactional 1 usage 2 tjoeunJava +1
public void updateApplyStatus(Integer applyHistoryId, String newStatus) {
    ApplyHistory applyHistory = applyHistoryRepository.findById(applyHistoryId)
        .orElseThrow(() -> new IllegalArgumentException("지원 내역을 찾을 수 없습니다."));

    ApplyHistory.ApplyStatus status = ApplyHistory.ApplyStatus.valueOf(newStatus);
    applyHistory.setStatus(status);
    applyHistoryRepository.save(applyHistory);
    applyHistorySearchService.updateStatusInES(applyHistoryId, status);
}
```

AdminService

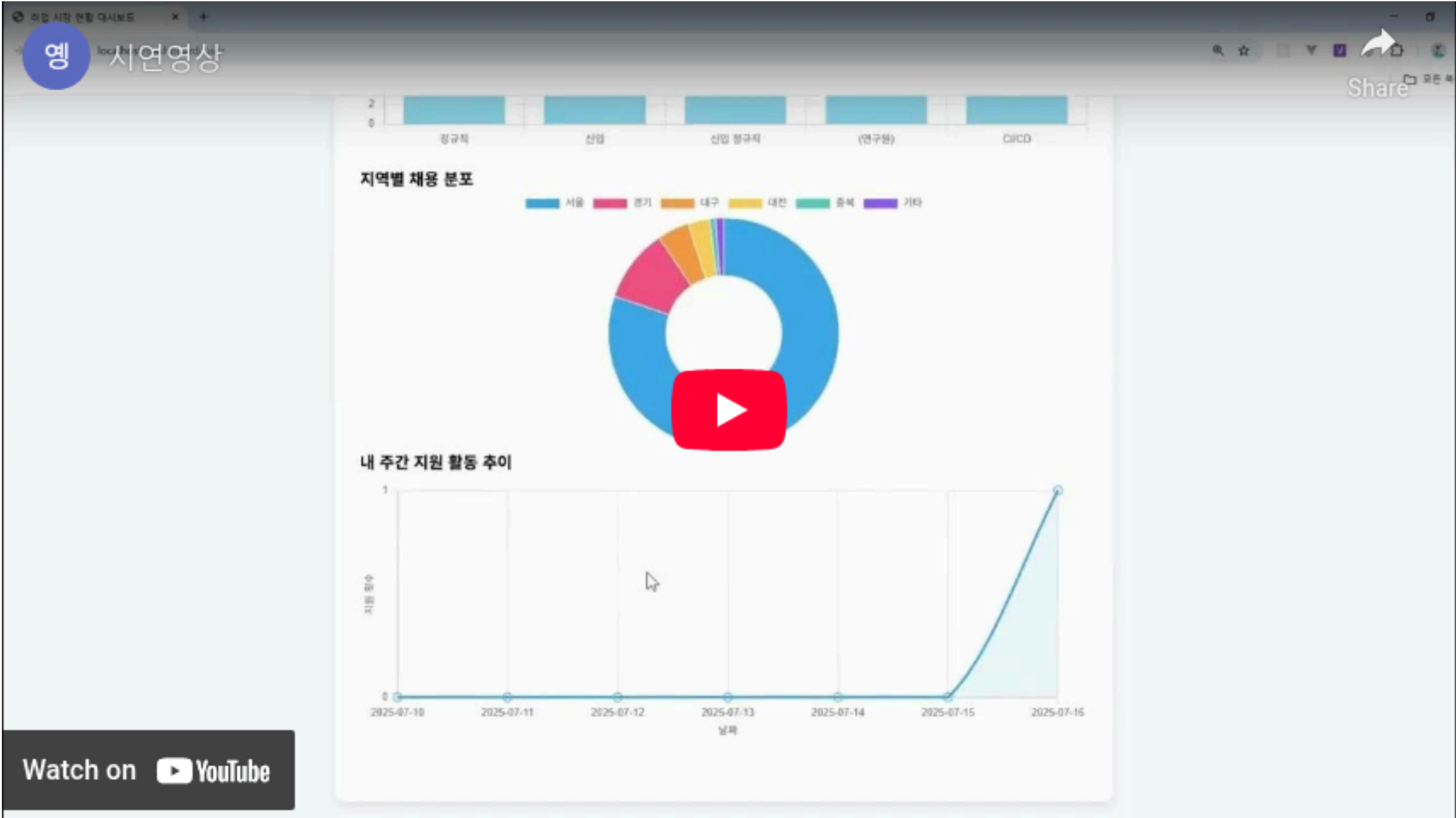
```
if (applyPage.isEmpty() && page > 1) {
    page = page - 1;
}

redirectAttributes.addAttribute(attributeName: "statusChanged", statusDisplay);
redirectAttributes.addAttribute(attributeName: "page", page);
redirectAttributes.addAttribute(attributeName: "sortOption", sortOption);

if (title != null) redirectAttributes.addAttribute(attributeName: "title", title);
if (company != null) redirectAttributes.addAttribute(attributeName: "company", company);
if (email != null) redirectAttributes.addAttribute(attributeName: "email", email);
if (nickname != null) redirectAttributes.addAttribute(attributeName: "nickname", nickname);
if (status != null) redirectAttributes.addAttribute(attributeName: "status", status);
if (startDate != null) redirectAttributes.addAttribute(attributeName: "startDate", startDate);
if (endDate != null) redirectAttributes.addAttribute(attributeName: "endDate", endDate);

return "redirect:/admin/apply_detail/" + applyHistoryId;
}
```

4. 프로젝트 수행 경과



5. 자체 평가 의견

✓ 완성도 평가

- 크롤링, 전처리, 검색, 대시보드, 챗봇까지 실질적인 서비스 구조를 단계별로 완성
- 관리자·사용자 기능을 분리하여 실제 서비스처럼 전체 시스템 흐름 구현
- 일부 검색 정확도, UI 정교화, 추천 시스템 등은 시간 제약으로 미흡

✓ 장단점 및 추후 보완 계획

- 크롤러를 직접 구현하고 Elasticsearch 연동까지 구성한 점은 기술적으로 높은 완성도
- 실사용자 중심의 기능 설계와 Kibana 기반 데이터 시각화는 활용성과 확장성 측면에서 우수
- 향후에는 사용자 행동 기반 추천, 자동완성 검색, 이력서 분석 등 고도화 기능 강화 계획

5. 자체 평가 의견

프로젝트를 수행하면서 느낀 점이나 경험한 성과(경력 계획 등과 연관)

✓ 프로젝트 후기

이예영

이번 통합 채용 공고 검색 및 지원 관리 플랫폼 개발을 통해 Elasticsearch와 Spring Boot 기반의 데이터 처리 및 검색 최적화 역량을 키웠으며, 개발자로서의 문제 해결 능력과 경력 확장에 중요한 기반이 되었다고 느꼈습니다.

권현경

이번 프로젝트를 통해 크롤링 및 Elasticsearch를 활용한 검색 기능 구현 과정을 직접 경험할 수 있었고, API 설계와 Spring 프레임워크를 보다 세부적으로 이해하고 응용하는 방법을 배울 수 있었습니다. 좋은 실습 경험이 되었습니다.

안보현

Spring Boot와 Elasticsearch를 활용해 실시간 검색 시스템을 구현하며 전 과정을 경험하며 실무 감각을 익혔습니다. 이로써 백엔드 개발 역량을 키울 수 있었습니다. 성장하는 개발자로 나아가는데 중요한 발판이 되었습니다.

이가은

프로젝트에서 크롤링을 통한 데이터 수집 시스템을 구축하며 효율적인 데이터 처리와 실시간 수집 경험을 쌓았습니다. 또한, 팀원들과의 협업을 통해 문제 해결 능력을 강화하고, 백엔드 개발자로서 중요한 역할을 할 자신감을 얻었습니다.

더조은 컴퓨터 아카데미 (종로)

경청해주셔서 감사합니다.

팀원 | 이예영 권현경 안보현 이가은

멘토 | 장기태